electronics

MDPI

Article

AI-Driven Rule-Based Feature Scoring for Explainable and Adversarially Robust Android Malware Detection

Assem Alhawari [1] and Sahar Ebadinezhad [1,2,*]

[1] Department of Computer Information Systems, Near East University, Near East Boulevard, 99138 Nicosia, Northern Cyprus (TRNC), Turkey ; 20214873@std.neu.edu.tr
[2] Computer Information Systems Research and Technology Center (CISRTC), Near East University, Near East Boulevard, 99138 Nicosia, Northern Cyprus (TRNC), Turkey
* Correspondence: sahar.ebadinezhad@neu.edu.tr

Abstract

The rapidly evolving Android malware that employs obfuscation and adversarial techniques has become a critical challenge for cybersecurity detection systems. This study introduces RFS-MD, short for Rule-based Feature Scoring for Malware Detection, a framework that builds explainability and adversarial defense into the detection pipeline by deriving feature importance scores directly from classification association rules. Experiments across several machine learning (ML) and deep learning (DL) classifiers on a balanced static feature dataset demonstrate that scored features improved recall across all six evaluated classifiers under cross-validation, with statistically significant accuracy and recall gains confirmed in four of the six. Under-tuned hyperparameter configurations, scored features consistently outperformed non-scored features across all classifiers. Furthermore, RFS-MD enhanced model robustness against adversarial attacks, reducing attack success rates and maintaining positive recall gains over baseline models. A rule-based explainability approach (RXAI) is introduced to generate transparent and human-readable model explanations; fidelity analysis confirms that RXAI captures interacting malicious feature patterns aligned with classifier decisions. Overall, the findings show that rule-based feature scoring offers a single, unified way to tackle accuracy, robustness, and explainability in Android malware detection simultaneously, contributing to trustworthy AI-driven cybersecurity solutions.

Keywords: Android malware detection; explainable AI; artificial intelligence; adversarial attacks; security; cybersecurity; machine learning; deep learning; feature scoring; association rules

1. Introduction

In recent days, Android has become one of the most widespread and used operating systems in many smart systems and devices that we use in our daily lives. Smartphones, the Internet of Things, smart home devices, wearable technology, and automotive systems have become an important part of our daily routine that is indispensable [1]. Consequently, malware attacks and cyber threats have increased, targeting users' personal information, financial accounts, and more. Malware developers have used several techniques to hide malicious characteristics and behaviors in their applications to achieve their goals. These techniques include code hiding, polymorphism, and dynamically loading payloads to circumvent traditional security measures [2,3]. In addition, they used artificial intelligence (AI) algorithms to enhance the success of malware concealment and undetectability, rendering traditional detection methods, such as signature-based approaches, ineffective. This

Check for updates

Received: 19 April 2026
Revised: 13 May 2026
Accepted: 14 May 2026
Published: 18 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

reality has led to a significant reliance on machine learning and deep learning techniques, which can automatically learn the behavioral patterns of malware and are more effective in the detection process [4].

Machine learning (ML) and deep learning (DL) techniques are now implemented to detect malware effectively; the malware classification process is one of the major safety precautions that serve to counter malware threats in many cybersecurity domains. It provides vital insights and enables proactive defense. Two types of malware classification systems are implemented by ML-based defense mechanisms: binary classification, in which ML and DL models classify each example as malware or benign, a process known as malware detection, and multi-class classification, in which ML models perform multiple classifications for a set of examples, in which they can classify the malware into its family [5]. It is essential at the outset to review the type of features extracted from software examples for the malware classification process. This process is essential for training ML classifiers because the performance of the classifier depends on the feature types extracted and the amount of information they carry to support classification results [6]. The APK files of the Android application include DEX and manifest files that contain the main features, which are analyzed and extracted. APK file features are either static features, which are extracted from the file without executing the internal commands and instructions of the file; it is safe when it comes to extracting features from the malware [7], such as API calls, function imports, file header analysis, code structure, string analysis and the manifest file, or dynamic features that are extracted only if the program has been executed in a monitored environment to evaluate malware behavior, usually by executing it in a sandbox or virtualized environment, such as system calls, network traffic and registry features [8].

Nowadays, attackers work on developing inverse counter methods and try to be in advance of the designed defense systems, such as developing adversarial attacks based on manipulating input features and classification results. The process of modifying malware features requires adherence to certain conditions to ensure the success of the camouflage in deceiving the classifiers [9]; for instance, the malware should remain operable and functional after modification to carry out its duty and without noticeable changes in the external appearance of the malware to reduce the chance of it being detected by the classifiers, such as maintaining the size of the file unchanged after the modification [10].

Despite the effectiveness of ML and DL models in malware detection, their inherently opaque decisions represent a very critical issue, particularly in cybersecurity environments where analysts must understand the malicious behavior of the detected malware by the model and act based on that. This transparency gap has motivated the researchers to involve interpretability techniques and explainable artificial intelligence (XAI) models in malware detection models that faced adversarial attacks. These explainable models are categorized into different types. For example, model-agnostic or model-specific. Model-agnostic is the XAI models that are not limited to specific prediction algorithms such as SHapley Additive exPlanations (SHAP) [11] and Local Interpretable Model-agnostic Explanations (LIME) [12]. While model-specific means the XAI model is only used for a specific detection model because its own method in explanations associates with how the prediction model works such as Gradient-weighted Class Activation Mapping (Grad-CAM) [13]. In addition, XAI models may be classified into local and global models; local models are used to explain the individual prediction from the model results such as LIME, which illustrates the malicious features that contribute to classify a specific sample to a malware, while global ones identify the effective malware features across the whole behavior of the malware detection model [14]. On the other hand, XAI models could also be divided into post hoc and intrinsic models; post hoc models explain the models after they make decisions, while intrinsic

models include Sparse Linear models and decision trees (DTs), which are simple and have a self-explained structure [15].

The paper's main contributions are the following:

1. A novel rule-based feature scoring framework for malware detection (RFS-MD). It is a unified framework that combines rule-based importance feature scores with rules derived from the FP-growth algorithm to identify and quantify meaningful feature interactions in Android applications. The extracted rules are used to assign importance scores to features, which serve as inputs to ML and DL classifiers, embedding feature interpretability into the learning process to ensure the explanations of the model's decisions truly reflect the influence of the features on the classifier predictions.
2. Improved detection performance with emphasis on recall. RFS-MD consistently improves the malware detection performance in ML and DL models, with stability in recall gains. Recall is considered a critical performance metric in security applications, for which it is used to evaluate the defense system's ability to discover malware samples correctly to avoid significant security risks.
3. A unified framework for performance, robustness, and explainability, RFS-MD bridges the gap between critical issues in the field of Android malware detection through using a rule-based feature scoring technique with the extracted rules.
4. Enhanced robustness against adversarial attacks. The proposed framework exhibits malware detection robustness against white-box and black-box attacks. It improves the resistance compared to baseline models; it highlights important features using rule-based feature scoring to reduce attack success rates.
5. Reliable and high-fidelity rule-based explainability mechanism (RXAI) integrated within RFS-MD. Our proposed RXAI provides a human-readable explanation of the model decisions that capture interacting feature relationships that align with underlying classifier results. Unlike traditional XAI models, RXAI does not focus on individual features; fidelity confirms that RXAI accurately reflects the underlying model decisions, particularly for malware features.
6. Compatibility with both ML and DL classifiers. RFS-MD with RXAI works as a model-agnostic and post hoc approach with intrinsic capability; it includes scored feature representations as structured inputs to several classifiers while still allowing post hoc explanation through extracted rules, demonstrating model flexibility and applicability without changing the architecture of the classifiers.

Despite the progress made in ML and DL-based malware detection, three closely connected problems remain unsolved in the literature. The first is the persistent trade-off between explainability and performance: models designed to be interpretable tend to give up detection accuracy, while high-performing black-box models produce explanations that are post hoc approximations with no real connection to how the model actually learned [16,17]. The second is that widely used XAI methods, including SHAP, LIME, and gradient-based approaches, treat features as independent contributors and miss the combinatorial co-occurrence patterns between features that often define malicious behavior in Android applications [18,19]. The third is that no existing framework tackles high detection recall, adversarial robustness, and high-fidelity explainability together through a single shared mechanism. Studies chasing higher accuracy tend to use opaque models with no explainability component, studies proposing XAI mechanisms typically test them on clean data only, and adversarial robustness work rarely brings explainability into the picture at all. The proposed RFS-MD framework addresses all three gaps directly by embedding classification association rule confidence values into the input feature representation itself, making feature importance part of the learning process from the start rather than something layered on after training is conducted.

The rest of the paper is organized as follows. Section 2 explores important related works and the gaps found in the literature. Section 3 presents the methodology of our proposed RFS-MD framework with all phase details. Section 4 shows the experiments conducted with several ML and DL classifiers and their results in terms of the classifier's performance over the scored and non-scored features, alongside robustness evaluation experiments and the explainability results. Section 5 introduces the results discussion, whereas Section 6 summarizes the Conclusion and future works.

2. Literature Review

Android devices have increased significantly among users due to their open-source nature and spread in the global market. This led to the most popular targets for attackers who are adopting the evolution of cyber threats. Reports indicate that millions of Android malware are developed yearly, which are used to collect user data and exploit software security gaps [20]. Several malicious activities occur without users' awareness, such as unauthorized access, data exfiltration, and financial fraud. Despite the protection and detection built into the security system, adversarial attacks continue to bypass these defense mechanisms using new evasion techniques and code obfuscation [21]. The traditional techniques of malware detection rely on signatures by identifying known patterns. Despite the effectiveness of these methods in detecting malicious behaviors, they cannot detect zero-day attacks or modifications to the code structure. These limitations lead to the adoption of more advanced detection methods, such as behavior-based and anomaly-based techniques, which aim to detect abnormal changes in app behaviors [22]. This arms race between attackers and defenders creates new adaptive and intelligent techniques for malware detection and pushes researchers toward approaches that use large-scale, evolving data [23].

Machine learning algorithms can handle large datasets and generalize to new threats, unlike traditional defense systems that depend on signatures. ML algorithms can analyze malware behavior and detect zero-day attacks using app features such as API calls, intents, and permissions. These approaches are supervised learning algorithms that are trained on malware features datasets to build a detection model, including RF, SVM, KNN, and DT [20,21]. Moreover, these ML models depend on feature engineering despite their high performance and may not adapt rapidly to malicious features without frequent training [24]. Deep Learning approaches were performed in the malware detection field to bypass the limitations of traditional ML models, with respect to their ability to identify a high level of malicious behavior, such as CNN, LSTM, RNN, and FNN. They achieve high performance by capturing the nonlinear relationships between the features and the class label [25]. However, DL models are black-box models, which are considered very challenging for transparency and trust, and understanding the logic behind their decisions and predictions [26]. On the other hand, recent studies show that ML and DL models are vulnerable to adversarial attacks through which smart systematic feature perturbations can change the classifier's prediction towards a different class compared with the predictions before the attack [27].

Two structural problems persist across classifier types in the reviewed literature. The first is that detection performance remains tightly tied to the input feature representation: squeezing out more accuracy tends to demand either a more complex architecture or richer feature engineering and rarely both at once without serious computational cost. The second is that no single classifier manages to deliver high recall, strong adversarial robustness, and meaningful explainability together, all three of which are non-negotiable in any detection system built for real-world deployment [28,29].

The black-box nature of ML and DL classifiers has made explainability a practical necessity rather than an academic preference in security applications. Analysts cannot act on a classification verdict alone; they need to know which specific behaviors or features drove the decision in order to respond, build remediation plans, and develop broader threat intelligence [30]. SHAP and LIME have become the dominant tools for meeting this need. Both are model-agnostic, meaning they attach to any trained classifier without requiring changes to its architecture, and both operate at the level of individual predictions by measuring how much each feature shifted the model's output for that specific sample [31–33]. Their flexibility has driven widespread adoption across malware detection research, in which they have been used to surface the API calls, permissions, and intents that most strongly influenced specific detection decisions, giving security teams at least a partial window into what the model was responding to [11,12].

Broad adoption has not, however, resolved the fundamental questions about whether these explanations are reliable. Three limitations run consistently through the SHAP and LIME literature. The explanations are local, they characterize one prediction at a time and say nothing about the model's systematic behavior across the dataset [34]. They are unstable; small shifts in the feature space or differences in model architecture can produce meaningfully different attributions for the same application under different conditions [32]. And most critically for malware analysis, they treat every feature as an independent contributor, which means they can report that permission A is important and API call B is important, but they say nothing about what it means when A, B, and C appear together, the kind of joint pattern that often defines malicious intent rather than any single signal in isolation [16].

Rule-based approaches using association rule mining offer a direct answer to this last limitation. By expressing explanations as antecedent–consequent relationships mined from training data, methods built on FP-Growth or similar algorithms capture exactly the co-occurrence patterns that SHAP and LIME miss, identifying which feature combinations collectively point toward malware or benign behavior rather than ranking features one by one [19,35]. They also support both local and global explanation simultaneously, something post hoc attribution methods cannot do without aggregation artifacts [36]. The trade-off is not eliminated, though; it just moves. Decision tree rules share the human-readable structure of association rules but are structurally fragile: minor perturbations in training data shift splitting points and produce different explanations on different runs, and DT classifiers as a group do not match the detection accuracy of ensemble or deep learning models [16]. The pattern that emerges from comparing these two families is telling. SHAP and LIME preserve accuracy but produce explanations that are post hoc approximations with no guaranteed alignment with the model's actual reasoning. Rule-based and DT methods produce explanations that are structurally grounded and human-readable but come with a measurable accuracy cost. Trillo et al. [37] put concrete numbers to this: in a systematic evaluation across cyber threat detection tasks, non-interpretable neural networks reached a macro F1-score of 0.8786, while the most explainable models achieved only 0.6106 under identical conditions. More transparency, in other words, costs roughly 17 percentage points of F1. A high level of explainability demands simpler, more constrained models; high detection performance demands the opposite [18]. Neither family resolves both requirements at once.

Robustness against adversarial manipulation adds a third dimension to the problem and one that intersects with both of the limitations above. Attackers targeting ML and DL malware detectors do not need to break the model outright; they only need to push a malicious sample across the decision boundary by selectively modifying its feature vector: adding permissions, inserting API calls, and adjusting intents, all while keeping the

malware's core functionality intact [38]. The degradation this causes is not minor. Studies consistently show that classifiers achieving strong clean-data accuracy can fail substantially against even moderately crafted adversarial examples [38]. Adversarial training, building perturbed samples into the training set so the model learns to recognize them, has produced measurable gains in robustness against the specific attacks it was trained on [2]. Feature selection and robust optimization have been proposed as complementary strategies, with the idea that concentrating the discriminative weight on features that are harder to suppress narrows the available attack surface [27]. Yet neither direction generalizes well: a model hardened against one attack type typically remains vulnerable to others, and as Slack et al. [39] showed, an attacker who understands the explanation layer can go further still, crafting classifiers that produce deliberately misleading SHAP and LIME attributions while continuing to misclassify malware, presenting a plausible-looking explanation surface that conceals what the model is actually doing. This is where the adversarial and explainability problems converge. When an input has been adversarially perturbed, there is no guarantee that the features flagged by a post hoc explanation method still correspond to what the model was actually responding to. The explanation may remain visually coherent while pointing entirely in the wrong direction. How explanation fidelity behaves under varying levels of adversarial perturbation has received almost no systematic attention in the reviewed literature, and it is precisely the scenario where a security analyst is most likely to rely on an explanation to understand what they are looking at.

The recent literature lacks a comprehensive model that effectively integrates high-quality, semantic rule-based explanations with high performance for the malware detection model, eliminating the trade-off between explainability and model performance while increasing model robustness against adversarial feature-based attacks. Post hoc feature attribution methods and rule-based explanation approaches have each moved the field forward in different ways, and the adversarial defense literature has produced real improvements in model robustness. The result is a field where each improvement is local: it solves one problem while leaving the other two open. The gap is not a matter of refinement within any existing approach; it is a question of whether the three properties can be addressed through a single shared mechanism rather than three separate additions to the pipeline.

In this study, we present a rule-based feature scoring for a malware detection model (RFS-MD), which addresses the gaps and limitations in the literature regarding model robustness against adversarial attacks and malware detection performance; it introduces the feature importance scores derived from the rules as a new feature representation to enhance model performance and robustness, and these importance scores identify the strength of the classification's contributions of the individual features within specific samples. In addition, we propose a novel rules-based explanations method (RXAI) as a model explainability part of the RFS-MD model to address the previous model explainability gaps found in the literature by systematically integrating the extracted Classification Association Rules (CARs) from the FP-Growth algorithm outputs into a malware detection model to provide rule-based explanations associated with ML and DL model prediction results. The use of CAR rules for both feature scoring and model explainability indirectly maintains the explanations' fidelity under adversarial attacks.

3. Methodology

This section presents the methodology of our proposed RFS-MD framework applied in this work regarding ML and DL used under different configurations, adversarial attacks, and explainability. The goal is to develop a comprehensive framework to enhance the performance of explainable malware detection models and their robustness under adversarial attacks. The methodology begins with extracting static Android features from

APK files. The dataset was divided into a training dataset and a test set, with 85% and 15%, respectively, using a stratified split to preserve class distribution. On the training dataset, we reduced feature dimensionality using the mutual information selection method, preserving highly informative features. Class Association Rules (CARs) are generated using the FP-Growth algorithm. The CARs rules are used in the proposed rule-based feature scoring (RFS) approach to replace binary feature representations with scored feature representations based on rule confidence values. The scored features dataset is used to train several ML and DL classifiers to enhance their malware detection performance and to increase their robustness against adversarial attacks. In addition, CAR's rules are used to support the explainability of model decisions.

Figure 1 illustrates the main procedures of our proposed methodology. The general framework is summarized in Algorithm 1, while the details of each phase are described in the following subsection.

Algorithm 1 Proposed explainable malware detection framework.

Require: APK samples collected from AndroZoo repository
Ensure: Malware detection models, explainable rules, and adversarial robustness evaluation

Phase 1: Dataset Preparation and Feature Extraction
1: $Dataset \leftarrow$ Algorithm 2
Phase 2: Feature Selection using Mutual Information
2: $ReducedDataset \leftarrow$ Algorithm 3
Phase 3: Association Rule Generation using FP-Growth
3: $CAR \leftarrow$ Algorithm 4
Phase 4: Rule-Based Feature Scoring (RFS)
4: $(ScoredDataset, BinaryDataset) \leftarrow$ Algorithm 5
Phase 5: Models Training Including RFS-MD and Evaluation
5: $Models \leftarrow$ Algorithm 6
Phase 6: Adversarial Attack Evaluation
6: $Attacksuccessrate(ASR), RecallGain(RG) \leftarrow$ Algorithm 7
Phase 7: Rule-Based Explainability (RXAI) and Fidelity Evaluation
7: $explanations, Fidelity \leftarrow$ Algorithm 8
8: **return** *Models*, explanations, and adversarial robustness results

3.1. Dataset Preparation

The dataset was obtained from the Androzoo repository [40], which is publicly available, contains millions of Android applications from different sources such as play.google.com or appchina, and has been widely used in malware analysis and malware detection research [41]. In addition, it provides many options before downloading the datasets, including the number of VirusTotal (VT) engines used to flag the APK files as malware, APK size, APK date, scanning date, package name, and version code. Malware samples were selected based on a VirusTotal detection count above 30 antivirus engines, making sure only applications with strong, multi-source confirmation of malicious behavior were included. Benign samples were limited to applications with zero VirusTotal detections, giving a clean, unambiguous negative class. The final dataset is perfectly balanced with 4500 samples per class, which removes any class imbalance bias from the evaluation, covering multiple years up to 2024.

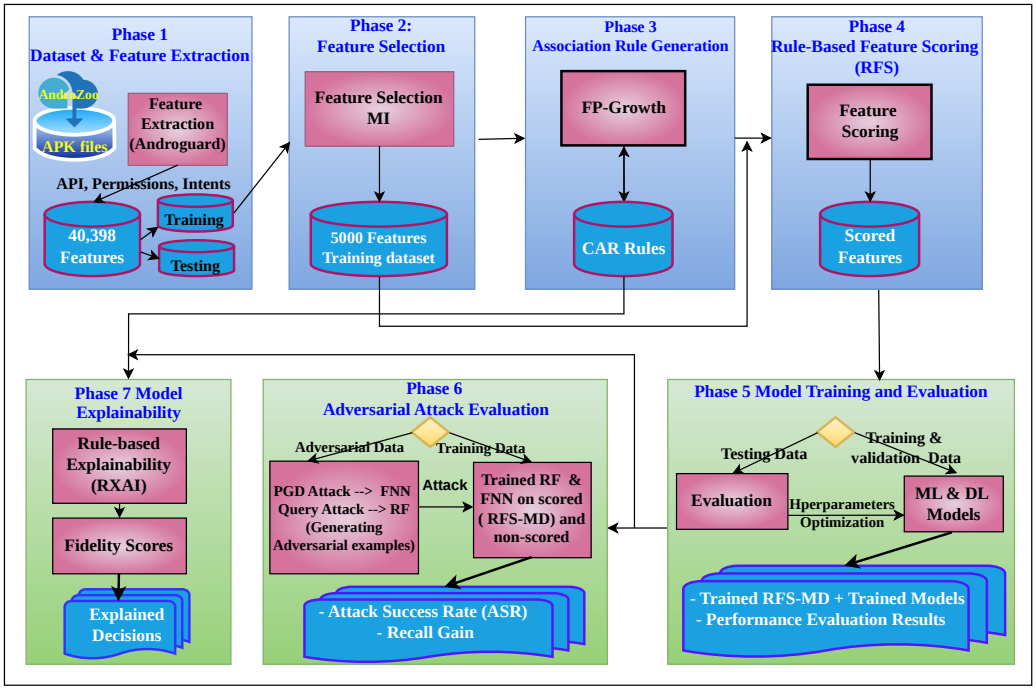


Figure 1. Workflow of our methodology.

Androguard toolkit handled feature extraction, turning each APK into a row in a 9000 x 40,398 binary matrix, including API call, permission, or intent features. A 1 means that a feature was present, a 0 means that it was not. The matrix is highly sparse, with only 1.15% of all entries being non-zero. This extreme sparsity is characteristic of Android static feature spaces, where individual applications draw on a small and specific subset of the total available API and permission vocabulary. Mutual information-based feature selection reduced the dimensionality from 40,398 to 5000 for the training set, retaining the most discriminative features, as indicated by a clear elbow in the MI score distribution, beyond which scores approach zero. This reduced set forms the baseline dataset, referred to throughout this study as the non-scored feature dataset, in which malware is assigned a positive label of 1 and benign a negative label of 0. Every APK in the dataset is identified by its SHA256 hash as assigned by AndroZoo, which means the exact sample set can be reconstructed by anyone querying the repository with the same selection criteria, keeping the experimental setup fully reproducible.

A static feature analysis was chosen over dynamic analysis for three interconnected reasons. Static analysis is computationally efficient and scalable to large-scale deployment without requiring dynamic execution environments, making it safe and scalable. Second, the balanced dataset of 9000 applications with consistent static feature extraction allows for fair, reproducible evaluation across all seven classifiers. Third, the contribution of the paper is feature-type-agnostic, meaning that the RFS-MD framework could, in principle, be extended to dynamic or hybrid feature sets in future work. Furthermore, static features remain the most practical starting point for large-scale detection studies and reflect the design priorities of oriented analysis for deployment. The dataset preparation and feature extraction are described in Algorithm 2.

Algorithm 2 Dataset preparation and feature extraction.

Require: APK samples downloaded from AndroZoo
Ensure: Binary feature dataset
MalwareSamples ← APKs with VirusTotal detection > 30
BenignSamples ← APKs with VirusTotal detection $= 0$
n ← total number of APK samples (9000)
for $i = 1$ to n **do**
    $(Manifest_i, Dex_i) \leftarrow AnalyzeAPK(APK_i)$ using Androguard
    Extract $APIs_i$, $Perm_i$, Int_i from $Manifest_i$ and Dex_i
    for each feature f in *Feature_Set* **do**
        if f exists in APK_i **then**
            $FeatureVector_i[f] \leftarrow 1$
        else
            $FeatureVector_i[f] \leftarrow 0$
        end if
    end for
    if $APK_i \in MalwareSamples$ **then**
        $Label_i \leftarrow 1$ ▷ 1 represents malware
    else
        $Label_i \leftarrow 0$ ▷ 0 represents benign
    end if
end for
Dataset ← combine all $(FeatureVector_i, Label_i)$
return Dataset

3.2. Feature Selection and Dimensionality Reduction

The baseline dataset is sparse. We applied the MI algorithm to reduce the dimensionality of the features and select the most informative features that have a high dependency on the class label, keeping the features that contribute more to building a classification model, as shown in Equation (1) [42].

$$MI(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log\left(\frac{p(x,y)}{p(x)\,p(y)}\right) \quad (1)$$

X symbolizes static features, and Y represents the class label (malware or benign). $P(x,y)$ is the probability that x and y occur together.

The optimal number of features (k) in MI was determined by performing MI score distributions on all 40,398 features with the elbow method. As Figure 2 shows, a highly skewed pattern with an inflection point where the MI score drastically decreased after k = 5000. This indicates that a small number of features depend on the class label, while the majority of features are nearly independent or are noise. In addition, another experiment was applied with a minimum MI threshold value of 0.001 to remove all near-zero MI scores; the feature space was reduced to 5029. Based on these findings, the baseline dataset was reduced to 5000 features. This reduction preserved the most informative features that enhance model performance and decreased model complexity, computational cost, and memory consumption, which is critical for large-scale malware analysis. Features selection process using MI is illustrated in Algorithm 3.

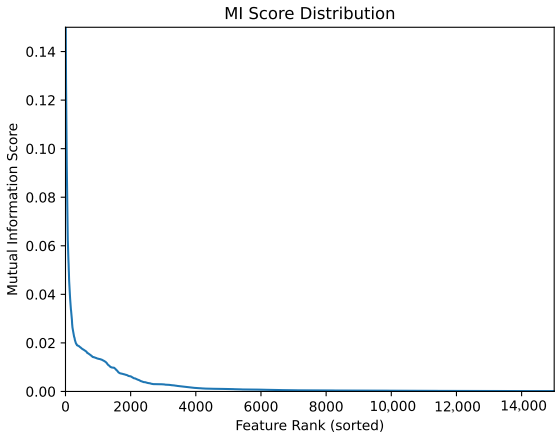


Figure 2. MI scores' distribution.

Algorithm 3 Feature selection using mutual information.

Require: Binary feature dataset *Dataset*
Ensure: Reduced feature set *ReducedDataset*, *TestSet*
1: Split *Dataset* into TrainingSet and TestSet
2: Compute $MI(f, Label)$ for each feature f in *TrainingSet*
3: Plot MI scores to generate Elbow Diagram
4: $k \leftarrow 5000$ ▷ Elbow observed where MI drops sharply
5: Sort *FilteredFeatures* in descending order based on $MI(f, Label)$
6: *SelectedFeatures* ← top k features from *FilteredFeatures*
▷ $k = 5000$ most informative features
7: *ReducedDataset* ← *TrainingSet* restricted to *SelectedFeatures*
8: **return** *ReducedDataset*, *TestSet*

3.3. Rule Generation

The overall processing pipeline for rule generation is illustrated in Algorithm 4. First, the Fp-Growth algorithm was applied on the selected features to generate frequent itemsets with a minimum support value (minsup) threshold of 0.01; each itemset produced includes several related features. The minimum support threshold was set to 0.01, meaning that only feature co-occurrence patterns appearing in at least 1% of training samples were kept. This value came out of preliminary experiments on the training data that tested values of 0.005, 0.01, and 0.02. Going below 0.005 triggered a flood of itemsets built on rare co-occurrences with shaky confidence estimates, while pushing above 0.02 cut out infrequent but genuinely useful malicious specific patterns. The chosen value sits at a balance that has solid grounding in the ARM literature: Han et al. [43] showed that, in sparse, high-dimensional transaction databases, a low minimum support is needed to surface meaningful co-occurrence patterns that would otherwise get thrown away, and Lawal et al. [44] confirmed that choosing an appropriate threshold must align with dataset sparsity to avoid either rule explosion or excessive pruning.

FP-growth algorithm discovers the hidden pattern and the relationships between features (items) [43], reducing the cost of searching time and memory consumption compared to other methods of Association Rule Mining (ARM) like the Apriori algorithm, which makes it the best choice to mine the rules for large high-dimensional datasets [44]. Minsup is used to filter out the itemsets with weak frequencies, as shown in Equation (2); the

support of Itemset (A, B) represents the percentage of samples that include features A and B over all samples in the dataset (D).

$$\mathrm{Sup}(A, B) = \frac{|S_{A,B}|}{|D|} \tag{2}$$

The second is to mine rules; rules in ARM are mined from each itemset, considering all possible combinations of the items (features) to shape the items of the left-hand side of the rule (antecedent) and the right-hand side of the rules (consequent) [45]. Each mined rule has its own confidence value, which is calculated as shown in Equation (3).

$$\mathrm{Conf}(A \Rightarrow C) = \frac{\mathrm{Sup}(A \cup C)}{\mathrm{Sup}(A)} \tag{3}$$

where A and C could be one feature or a subset of features from the related mined itemset. Confidence values express how strongly the rule is likely to happen [46]. If the confidence of a rule was 80%, when the features in A are present in a specific sample, it is most likely with 80% that features in C are also present in the same sample. Since the confidence value is calculated based on item frequencies over the whole dataset, the rule provides global feature relationships, not a feature-sample relationship, and this is another reason for choosing rules generation from ARM.

We only filtered the rules called Classification Association Rule (CAR) [19]. In this type of rule, the consequent is either malware or benign. To produce CAR rules, we add malware and benign features to the dataset with a binary value indicating the presence of these features in the samples. For example, if the sample is malware, in the "malware" feature column, the feature has a value of 1, and the "benign" feature column has a value of 0. So, it becomes a part of the itemset when generated by the FP-Growth algorithms. In the CAR Rule, confidence demonstrates how likely the rule is to happen between the samples of malware or benign, reflecting the presence of antecedent rule features within malware samples or benign samples. Only CAR rules are kept in a separate dataset, where they are used in two main phases of our proposed framework. Rule-based scoring phase and explainability phase.

The second filtering process is based on rule confidence threshold. Rules with confidence below 0.7 were left out of the scoring process; this threshold came from a preliminary evaluation. Rules with Low confidence values were found to add noise by assigning importance values to features whose link to the class label was weak and inconsistent, which hurt classifier performance rather than helping it. Put simply, a confidence of 0.7 means that, whenever a rule's antecedent features appear together in a training sample, they point to the same class label at least 70% of the time. That level of consistency is what makes the association worth keeping. This connects directly to how confidence works in association rule mining: it estimates the conditional probability P(class | antecedent), and drawing a minimum threshold around it is what separates rules that genuinely signal something from those that just got lucky in the data [43]. The rule generation phase is summarized in Algorithm 4.

Algorithm 4 Association rule generation using FP-growth.

Require: ReducedDataset with selected features
Ensure: Set of Association Class Rules *Rules*
Transform *ReducedDataset* into transaction format
$MinSupport \leftarrow$ predefined minimum support threshold
$MinConfidence \leftarrow$ predefined minimum confidence threshold
$FrequentItemsets \leftarrow$ FP-Growth mining on FP-Tree
for each itemset I in $FrequentItemsets$ **do**
  if I contains class label $\{malware\}$ or $\{benign\}$ **then**
    $Antecedent \leftarrow I - ClassLabel$
    $Consequent \leftarrow ClassLabel$
    Compute rule confidence:
    if $Conf \geq MinConfidence$ **then**
      $Rules \leftarrow Rules \cup (Antecedent \Rightarrow Consequent)$
    end if
  end if
end for
return *Rules*

3.4. Rule-Based Feature Scoring

The rule-based feature scoring (RFS) procedure operates on a per-sample basis and follows a two-path algorithm. For each sample, Si, is a primary scoring path (rule-matched features): First, candidate rules are identified: these are all CARs whose entire antecedent feature set is present in Si. Candidate rules are grouped by antecedent size (|antecedent|) and sorted in descending order within each group by confidence. A greedy selection procedure then processes groups from largest to smallest: for each group, rules are selected in descending confidence order, subject to a disjoint–antecedent constraint, which means that a rule is selected only if none of its antecedent features has already been assigned a score by a previously selected rule. Once a rule is selected, all features in its antecedent are assigned the rule's confidence value as their score, and those features are marked as 'used'. This continues until all groups have been processed. The disjoint constraint ensures that each present feature receives exactly one score, from the single most informative rule that covers it. There is no conflict resolution problem: the priority ordering guarantees that each feature is claimed by the first (highest-priority) applicable rule, and no subsequent rule can override that assignment.

Fallback scoring path (unmatched features), Features that are present in the sample (original value = 1) but not covered by any selected high-confidence rule are handled by a fallback strategy. Each such feature, f, is assigned the score max(P(malware | f), P(benign | f)), where P(malware | f) = support(f ∩ malware) / support(f) and P(benign | f) = support(f ∩ benign) / support(f), computed from training data frequencies. These probabilities are cached after the first computation and reused across samples to avoid redundant recalculation. The fallback score ensures that no present feature is left without a class-relevant signal, even though there are very few, and no high-confidence rules cover it. Absent features (original value = 0) are never matched to any rule and remain at zero in the scored representation. The scored feature vector, therefore, contains values in three ranges: zero for absent features, a fallback score in (0, 1.0] for unmatched present features, and a rule-confidence score in [0.7, 1.0] for rule-matched present features. This structured value range provides classifiers with richer information than a binary 0, 1 encoding: a feature scored 0.97 is associated with its class nearly unconditionally, while a feature scored 0.71 carries a weaker but still statistically meaningful signal.

The resulting dataset preserves the original features with additional information, which are represented by feature scores that reflect their importance and how much they

are related to the class label. The rule consequent (malware or benign) is not used during scoring, which prevents label leakage when producing the scored features dataset. At the end of this phase, two training datasets are produced: the dataset generated after the feature selection phase, which we refer to as the non-scored features dataset (baseline dataset), with binary features. The scored feature dataset was generated by our proposed rule-based feature scoring (RFS) technique. Both datasets were used to train the ML and DL models under the same experimental conditions and parameters. The rule-based feature scoring technique is illustrated in Algorithm 5.

Algorithm 5 Rule-based feature scoring (RFS).

Require: Binary dataset D, Classification Association Rules $\mathcal{CAR}$, Training counts: support(f), support$(f, \text{malware})$, support(f, benign)
Ensure: ScoredDataset, BinaryDataset

BinaryDataset $\leftarrow D$
fallback_cache $\leftarrow \emptyset$
for each sample S_i in D **do**
  SelectedRules $\leftarrow \emptyset$; UsedFeatures $\leftarrow \emptyset$
  PresentFeatures $\leftarrow \{f \mid S_i[f] = 1\}$
  // Step 1: Rule-based scoring (primary path)
  $\mathcal{CAR}_{\text{cand}} \leftarrow \{r \in \mathcal{CAR} \mid \text{antecedent}(r) \subseteq \text{PresentFeatures} \wedge \text{confidence}(r) > 0.7\}$
  Group $\mathcal{CAR}_{\text{cand}}$ by rule size $|\text{antecedent}|$
  for size z from largest **down to** 1 **do**
    Sort rules of size z by confidence (descending)
    for each rule r **do**
      if antecedent$(r) \cap$ UsedFeatures $= \emptyset$ **then**
        SelectedRules $\leftarrow$ SelectedRules $\cup \{r\}$
        UsedFeatures $\leftarrow$ UsedFeatures $\cup$ antecedent(r)
      end if
    end for
  end for
  for each rule r in SelectedRules **do**
    for each feature f in antecedent(r) **do**
      FeatureVector$_i[f] \leftarrow$ confidence(r)
    end for
  end for
  // Step 2: Fallback scoring for present but unmatched features
  UnmatchedFeatures $\leftarrow$ PresentFeatures $\setminus$ UsedFeatures
  for each feature f in UnmatchedFeatures **do**
    if $f \notin$ fallback_cache **then**
      if support$(f) > 0$ **then**
        conf_mal $\leftarrow$ support$(f, \text{malware})$ / support(f)
        conf_ben $\leftarrow$ support(f, benign) / support(f)
      else
        conf_mal $\leftarrow 0$; conf_ben $\leftarrow 0$
      end if
      fallback_cache$[f] \leftarrow \max(\text{conf_mal}, \text{conf_ben})$
    end if
    FeatureVector$_i[f] \leftarrow$ fallback_cache$[f]$ ▷ value in $[0, 1.0]$
  end for
  // Absent features remain 0 automatically
  Add FeatureVector$_i$ to ScoredDataset
end for
return ScoredDataset, BinaryDataset

The extra computational overhead of the proposed framework compared to the ML malware detection model is confined to an offline preprocessing phase, during which the high-confidence rules generation process is performed once on the selected features by the MI algorithm, and the CAR rules are stored for reuse. FP-Growth has high complexity, but its execution depends on the selected features and the minimum support threshold. The rule-based feature scoring (RFS) matches the sample features with the selected CAR rule dataset, in which the complexity is proportional to the number of rules and features, but this process is performed only once to produce a scored features dataset to train all ML classifiers. For deployment cases, only the lightweight scoring process is performed on new samples without using FP-Growth or retraining the models. So, the proposed framework involves a limited cost with maintaining suitability for real-time applications.

3.5. Experimental Setup

This subsection provides the experimental configurations adopted to evaluate the proposed RFS scoring method and assess robustness against adversarial attacks. Classification models were evaluated under the scored and non-scored feature representations using two complementary evaluation protocols: 10-fold cross-validation to assess the statistical reliability of performance differences, and a single held-out test set evaluation using optimized hyperparameters to report peak classification performance. Evaluation metrics were used to assess the malware detection performance with the same training and testing protocol for all types of models and parameter settings.

3.5.1. Classification Models

To evaluate the effectiveness of our proposed rule-based feature scoring technique (RFS), multiple widely used ML and DL models based on the literature were implemented. DT, KNN, RF, LR, SVM, CNN, and FNN [28,29]. DT is a tree-based classifier covering the feature vectors into tree partitions, presenting decision rules, while RF merges several decision trees for better accuracy and generalization performance. KNN is a distance-based classifier that assigns the new incoming instance the most common label based on the closest k samples. SVM finds the optimal decision boundary that maximizes the margin between classes. In addition to these previous classifiers, two neural network architectures were implemented as deep learning models. CNN was used to find feature patterns using the convolutional and pooling layers, while FNN contains multiple hidden layers to capture the high-level complex nonlinear features and learn to discriminate them for the classification process. This diversity of model types allows us to evaluate our approach under several malware detection models, including linear models, distance-based methods, ensemble learners, and deep neural networks.

3.5.2. Evaluation Metrics

The model's performance was assessed using standard evaluation metrics that include accuracy (Equation (4)), precision (Equation (5)), recall (Equation (6)), and F-score (Equation (7)); they were calculated based on the confusion matrix [47]. These metrics were selected because they are widely applied in malware detection and provide a comprehensive evaluation of classification behavior [16], such as accuracy, which is the percentage of correctly classified samples among all dataset samples. Recall value, which is the most important metric in this study, represents the percentage of actual malware that was successfully detected. A higher recall value indicates the system can discover most of the malware and does not miss any. While the precision value expresses how many of the

samples classified as malware are truly malware, low precision indicates that the system has many false alarms.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

$$Recall = \frac{TP}{TP + FN} \tag{6}$$

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{7}$$

where the true positives, TPs, are the number of malware samples correctly detected, and a high TP means the system is effective in finding threats. Meanwhile, the true negative samples, TNs, represent the number of benign samples correctly identified. A high TN reduces unnecessary user alerts. False positives, FPs, indicate benign apps incorrectly classified as malware. Too many FPs increase the false alarm, leading to user frustration and loss of trust. In contrast, false negative, FN, samples are malicious software incorrectly classified as benign. This is the most dangerous error, as undetected malware can compromise security. Additionally, we used the performance gain to demonstrate the improvement of using our proposed rule-based feature scoring across all models and parameter settings. The performance gain is calculated as the difference between the performance of the scored feature and the performance of the non-scored feature; if the result of the gain was positive, this indicates that using scored features demonstrates improvement [48]. The accuracy gain (AG) value is calculated as shown in Equation (8), and the recall gain (RG) value is calculated as shown in Equation (9), where S denotes the performance of the scored features, and NS represents the performance of the non-scored features.

$$AG = Acc(S) - ACC(NS) \tag{8}$$

$$RG = Rec(S) - Rec(NS) \tag{9}$$

3.5.3. Cross-Validation Evaluation Protocol

A 10-fold stratified cross-validation protocol was adopted for the primary performance evaluation. The cross-validation evaluation method assesses the statistical reliability of classification performance and addresses the variance associated with a single train–test split; the dataset is cut into 10 equal folds, with each keeping the same class balance as the full set. In each round, nine folds went to training, and one was held back for testing. Data leakage was avoided by running all preprocessing steps, including MI-based feature selection, FP-Growth rule mining, and RFS feature scoring, entirely within each training partition. Those parameters were then carried over to the test fold. Nothing from the test fold is fed back into preprocessing at any point. Each fold produced its own set of metrics; across all 10 folds, the mean and standard deviation were calculated for every classifier under both scored and non-scored conditions. To check whether the differences between conditions were real rather than random, a paired t-test was run on the 10 fold-level measurements, with $p < 0.05$ as the cutoff for significance.

3.5.4. Hyperparameter Optimization

In these evaluation experiments, each classifier was tuned to find the best parameters that yield the best performance. The dataset was divided once into a training partition (85%) and a held-out test partition (15%) using stratified sampling at the outset. All preprocessing operations, MI-based feature selection, FP-Growth rule mining, and RFS feature scoring were applied exclusively to the training partition, producing both the binary baseline dataset and the scored RFS-MD dataset. The test partition was withheld

from all preprocessing steps and was not accessed during any training or model selection procedure. Hyperparameter optimization was conducted by applying a 5-fold stratified cross-validation grid search on the original binary training set only, not on the scored representation. This ensures that the optimal hyperparameter configurations found during tuning are not influenced by the RFS-MD scoring mechanism, placing the baseline and the proposed method on equal footing. The best hyperparameters identified through this process were then applied to train classifiers on both the binary and scored training datasets for the tuned parameter evaluation.

For evaluation, the held-out test set was transformed using the pre-computed CAR rule dataset derived entirely from the training partition. The rules were not re-extracted or re-fitted using any test set information; the test set received the same deterministic scoring transformation applied to the training data using parameters computed exclusively from training samples. The test partition was accessed only once per experiment, for final performance measurement, ensuring all reported metrics reflect genuine out-of-sample generalization. Table 1 summarizes classifiers' key parameters that were explored during the grid search process to find the best parameters.

All sequential procedures of 10-fold cross-validation evaluation and tuned parameters evaluation are demonstrated in Algorithm 6.

Table 1. Hyperparameter search space used for model optimization.

Classifier	Hyperparameter Search Space
DT	max_depth: [None, 10, 20, 30, 40, 50], min_samples_split: [2, 5, 10, 20], min_samples_leaf: [1, 2, 5, 10]
KNN	n_neighbors: [3, 5, 7, 9], weights: [uniform, distance], metric: [euclidean, manhattan]
LR	penalty: [l1, l2], C: [0.01, 0.1, 1, 10, 100], solver: [lbfgs, liblinear], class_weight: [None, balanced]
RF	n_estimators: [100, 200], max_depth: [None, 20, 40], min_samples_split: [2, 5], min_samples_leaf: [1, 2], max_features: [sqrt]
SVM	C: [0.1, 1, 10], kernel: [linear, rbf], gamma: [scale, 0.01, 0.001]
CNN	filters: [32, 64], kernel_size: [3, 5], dropout: [0.3, 0.5], dense_units: [64, 128], learning_rate: [0.001, 0.0005], batch_size: [32, 64]
FNN	layers_config: [[512, 256, 128], [512, 256], [256, 128]], dropout: [0.1, 0.2], l2_reg: [0.001, 0.0005], learning_rate: [0.001, 0.0005], batch_size: [64, 32]

3.6. Adversarial Attack Configuration

To evaluate the robustness of our proposed malware detection models against adversarial applications, a framework for adversarial attacks was designed to simulate realistic adversarial examples. Two important clarifications regarding the semantic validity of the adversarial examples are warranted. All perturbations consist exclusively of 0 to 1 feature flips; features are only added to the malware sample's feature vector, never removed, which preserves the malware's existing API calls, permissions, and intents in their entirety and maintains the functional integrity of the original malicious application. Furthermore, neither attack selects features at random: the PGD attack identifies features whose addition most effectively shifts the model's output toward the benign class through gradient computation, and the query-based greedy attack selects feature additions that most reduce RF's malware confidence score at each iteration. Both optimization objectives structurally converge on benign-associated features, reflecting a realistic threat scenario in which an attacker inserts legitimate-looking code artifacts into a malicious application to conceal its malicious feature profile from static analysis. The perturbation process, therefore, guarantees fully functional adversarial samples by construction, and the evaluation

measures the structural resistance of the RFS-MD feature representation to this class of perturbation-based evasion attacks.

Algorithm 6 Model training and performance evaluation.

Require: BinaryDataset, ScoredDataset, FullDataset, TestSet
Ensure: Cross-validated and tuned performance results
CV_Classifiers ← {DT, KNN, LR, RF, NN, CNN}
Tuned_Classifiers ← CV_Classifiers ∪ {SVM}
▷ Configuration 1: 10-Fold Cross-Validation Evaluation
Partition *FullDataset* into 10 stratified folds
for each type T in {Binary, Scored} **do**
    for each classifier C in CV_Classifiers **do**
        for fold $k = 1$ to 10 **do**
            Compute MI, FP-Growth, and RFS on TrainFold_k only
            Train C on TrainFold_k; evaluate on TestFold_k
            Record Accuracy, Precision, Recall, F1-score
        end for
        Compute mean ± std; apply paired t-test (Scored vs. Binary)
        Store mean, std, p-value, and 95% CI for C
    end for
end for
▷ Configuration 2: Tuned Parameters Evaluation
for each dataset D in {BinaryDataset, ScoredDataset} **do**
    for each classifier C in Tuned_Classifiers **do**
        C_{best} ← Grid Search with 5-fold CV on D
        Train C_{best} on D; evaluate on TestSet
        Compute Accuracy, Precision, Recall, F1, Accuracy Gain, Recall Gain
    end for
end for
return CV statistical results and tuned performance results

We trained baseline models on the original binary features, and the RFS-MD models were trained on the scored features. This setup ensures identical training configurations and fair comparisons, allowing the evaluation to focus on the intrinsic robustness of the RFS mechanism. Therefore, any improvements in the RFS-MD model against the attack will refer to the enhanced features representation by the RFS technique and the model itself rather than using adversarial examples during training, confirming the robustness of the proposed approach in defending against unseen attacks.

The original dataset of the binary static features extracted from Android applications was divided into two datasets: one with 60% of the dataset to train our proposed RFS-MD model, and the baseline model. To train our RFS-MD model, the dataset must be scored using a rule-based scoring approach before being fed to the model. The other 40% of the dataset is used to generate adversarial examples and evaluate the models under the attacks of these examples. Several adversarial datasets were generated by attacking k features across all samples, where k represents the number of manipulated features. Increasing k will increase the strength level of the attack.

Two classifiers used in adversarial evaluation, RF and FNN, based on three considerations. First, they represent the two major adversarial threat paradigms, black-box query-based attacks for non-differentiable ensemble models, and white-box gradient-based attacks for differentiable neural networks; second, both models demonstrated the most consistent recall gains under clean conditions, making them the primary candidates for robustness analysis; and finally, applying attack-type-specific adversarial methods to all seven classifiers, while technically feasible, falls outside the primary scope of this framework evaluation and is reserved as a direction for future work. To generate adversarial examples, a project gradient descent (PGD) attack algorithm was conducted against FNN because NN is a differentiable classifier that depends on the gradient descent algorithm to optimize the classification performance, PGD is a white-box attack that computes gradients to modify effective features and generate adversarial examples [49]. While RF is non-differentiable, a gradient-based attack cannot be implemented directly. We used a query-based greedy feature attack as a black-box attack, through which it iteratively flips feature values from 0 to 1 while querying the model for the minimum predicted malware probability [50]. Applying this attack strategy allows us to evaluate how our proposed RFS-MD model enhances resistance under gradient-based attacks and query-based attacks.

We evaluate the model's resistance against the adversarial attack, using recall gain and attack success rate (ASR). The ASR is the percentage of malware samples that successfully evade model detection after attacking their features [28]; it measures the effectiveness of adversarial attacks. As shown in Equation (10), where $N_{success}$ denotes the number of successful adversarial examples, and N_{attack} is the total number of all malware samples.

$$ASR = \frac{N_{success}}{N_{attackk}} \times 100 \tag{10}$$

While recall gain measures the improvement in resistance of our proposed model, RFS-MD, compared with the baseline model's resistance. Adversarial Attacks methodology, including attack algorithms and evaluation metrics, is described in Algorithm 7.

3.7. Explainability Analysis

To enhance the explainability of the proposed malware detection model, we introduce a Rule-based Explainable Artificial Intelligence (RXAI) approach. In this phase, we use the Class Association Rules (CARs) dataset generated in Algorithm 4 to provide human-understandable explanations for each prediction. The proposed RXAI approach works as a post hoc agnostic model with Local and global explanations, focusing on rule-based explanations.

3.7.1. Rule-Based Explanations

To provide model decision explanations to our proposed model RFS-MD, for each sample, S_i, we identified the set of rules from the CARs dataset whose antecedent features are present in the relevant sample. The top K rules are selected based on the highest confidence values, starting from the largest size without any overlap with the features associated with the top K selected rules, following the same strategy that we used to select the rules for the scoring process. Then, the selected rules are used to provide human-understandable explanations for model classification decisions by showing the most influential interacting features that contribute to the model's decision and how confident they are associated with the class label, providing transparent and explained decisions.

Algorithm 7 Adversarial attack evaluation using feature-based attack

Require: D (original binary feature dataset), k_list (attack budgets), $classifiers = \{RF, FNN\}$, $scoring_rules$
Ensure: Evaluation metrics for each attack budget

Split dataset D
$TrainSet \leftarrow$ 60% of D
$AttackSet \leftarrow$ 40% of D
▷ Apply rule-based feature scoring
$Train_scored \leftarrow scoring(TrainSet)$
▷ Train RF and FNN on TrainSet and Train_scored
$RF_{scored} \leftarrow train(RF, Train_scored)$
$RF_{baseline} \leftarrow train(RF, TrainSet)$
$FNN_{scored} \leftarrow train(FNN, Train_scored)$
$FNN_{baseline} \leftarrow train(FNN, TrainSet)$
for each classifier C in $\{RF, FNN\}$ **do**
  for each attack budget k in k_list **do**
    for each malware sample x in $AttackSet$ **do**
      if $C = RF$ **then**
        $x_{adv} \leftarrow GreedyBlackBoxAttack(x, k)$
      end if
      if $C = FNN$ **then**
        $x_{adv} \leftarrow PGD_Attack(x, k)$
      end if
    end for
    $D_k^{adv} \leftarrow$ adversarial dataset generated with budget k
    ▷ Apply rule-based scoring on adversarial samples
    $D_{scored}^{adv} \leftarrow scoring(D_k^{adv})$
    ▷ Evaluate models
    $RF_{scored_results} \leftarrow evaluate(RF_{scored}, D_{scored}^{adv})$
    $RF_{baseline_results} \leftarrow evaluate(RF_{baseline}, D_k^{adv})$
    $FNN_{scored_results} \leftarrow evaluate(FNN_{scored}, D_{scored}^{adv})$
    $FNN_{baseline_results} \leftarrow evaluate(FNN_{baseline}, D_k^{adv})$
    ▷ Compute evaluation metrics
    Compute Attack Success Rate (ASR)
    Compute Recall Gain
    Store results for attack budget k
  end for
end for
return evaluation results

3.7.2. Fidelity Evaluation of Rule-Based Explanations

To evaluate the proposed RXAI model quantitatively, we proposed a fidelity approach to evaluate the effectiveness of model decision explanations. The fidelity metrics measure how much change in the model prediction values per sample occurs before and after removing the features involved in the top rules, evaluating the level of explainability of the top rules of the model. Higher fidelity means that the rules that are used to explain model decisions for the sample S_i include the most critical interacting features and relations that contributed to the class label, which lead to transparent and reliable explanations. As Equation (11) shows, the fidelity score is computed for each sample by the absolute value of the difference between the original prediction probability P_{original} and the modified prediction probability P_{modified} after removing the features of the model's top k explaining rules [51].

$$\text{Fidelity} = \left| P_{\text{original}} - P_{\text{modified}} \right| \tag{11}$$

The fidelity calculated at the sample level considers local fidelity. While the observed distributions and statistical measures (mean and median) provide an aggregated view of global fidelity, which reflects how well the extracted rule set collectively represents the overall behavior of the model across multiple samples. The proposed RXAI approach, including explaining rules and fidelity evaluation, is described in Algorithm 8.

Algorithm 8 Rule-based explainability RXAI and fidelity evaluation.

Require: Binary feature dataset D, Association Class Rules CAR, Trained model M, Number of rules k

Ensure: Explanations and fidelity scores for each sample

for each sample S_i in D **do**
  $SelectedRules \leftarrow \emptyset$
  $UsedFeatures \leftarrow \emptyset$ ▷ Select rules matching the sample
  $CAR_{candidate} \leftarrow \{r \in CAR \mid antecedent(r) \subseteq features(S_i)\}$
  Group $CAR_{candidate}$ by rule size
  for rule size z from largest to 1 **do**
    Sort rules of size z by confidence in descending order
    for each rule r **do**
      if $antecedent(r) \cap UsedFeatures = \emptyset$ **then**
        $SelectedRules \leftarrow SelectedRules \cup \{r\}$
        $UsedFeatures \leftarrow UsedFeatures \cup antecedent(r)$
      end if
    end for
  end for
  Extract contributing features:

$$F_i = \bigcup antecedent(r), \quad r \in \text{Top-}k \text{ rules}$$

  Compute original prediction:

$$P_{\text{original}} = M(S_i)$$

  Remove features F_i from S_i to obtain modified sample S_i'
  Compute modified prediction:

$$P_{\text{modified}} = M(S_i')$$

  Compute fidelity:

$$Fidelity_i = |P_{\text{original}} - P_{\text{modified}}|$$

  Generate explanation using selected rules and their confidence
end for
return Explanations and Fidelity Scores

4. Experimental Results

This section presents the experiments performed and the results obtained through this research on three main types of experiments, alongside the results analysis. First, the findings of the experiments obtained by evaluating the impact of Rule-based Feature Scoring on Malware Detection. Second, adversarial attack experiments to evaluate model robustness. Third, model decision explanations.

The codebase was written in Python 3.9. Scikit-learn 1.2 handled the ML classifiers, TensorFlow 2.10 with Keras took care of the neural network models, mlxtend was used for FP-Growth rule mining, and Androguard 3.3.5 did the feature extraction. The machine running all of this had an Intel Core i7, 32 GB of RAM, and a GPU. Random seeds were pinned across NumPy, Python's built-in random module, TensorFlow, and Scikit-learn before each run so the results could be reproduced exactly. The main detection experiments

used 10-fold cross-validation. Stratified sampling kept that class balance consistent in both halves. Hyperparameter tuning stayed inside the training split the whole time using 5-fold stratified cross-validation. The test set was never touched during this stage. Feature selection, rule generation, and scoring were all worked out on training data first, and then those same parameters were carried over to the test data. The adversarial evaluation used a separate 60/40 split of the full dataset, where the 40% evaluation partition provides a sufficiently large and diverse pool of samples for testing across multiple attack strength levels.

4.1. Impact of Rule-Based Feature Scoring on Malware Detection

This subsection evaluates the impact of rule-based feature scoring on malware detection performance through three complementary analyses: cross-validated statistical reliability assessment, peak performance under optimized hyperparameters, and a direct comparison against established alternative feature weighting methods.

4.1.1. Cross-Validated Classification Performance

Table 2 reports the 10-fold cross-validation classification performance for all six classifiers under scored (RFS-MD) and non-scored (baseline) conditions. Accuracy and recall are the primary reported metrics, with precision and F-score provided for completeness. The results show that four of the six classifiers, RF, LR, NN, and CNN, achieve statistically significant accuracy improvements under the scored representation. CNN shows the most pronounced gain, with mean accuracy rising from 97.01% to 97.72% ($p < 0.001$) and recall from 96.91% to 97.82% ($p = 0.0335$). The relatively higher standard deviation of CNN's baseline recall (±1.22%) compared to its scored recall (±0.66%) further indicates that the rule-based scoring not only improves the mean performance but also stabilizes the model's behavior across folds, a practically meaningful property for a security system where prediction consistency matters as much as peak performance. NN follows a similar pattern, with accuracy improving from 97.48% to 97.79% ($p = 0.0287$) and recall from 98.33% to 98.64% ($p = 0.0314$). For LR, accuracy improves from 97.05% to 97.27% ($p = 0.0393$) and recall from 96.51% to 96.89% ($p = 0.0415$), confirming that the continuous confidence score representation provides useful discriminative structure even for a linear classifier. RF achieves significant accuracy and recall improvements ($p = 0.0384$ and $p = 0.0412$, respectively), with the smallest standard deviations among all classifiers, reflecting the stability of the ensemble method under the scored representation.

DT and KNN show consistent positive directional improvements in recall, and positive but smaller gains in accuracy, none of which reach statistical significance at the 0.05 level. Notably, the positive recall direction holds across all six classifiers in the experiment, including DT and KNN, which is particularly meaningful in a security context where any consistent improvement in malware detection sensitivity carries practical value regardless of formal significance. DT partitions the feature space through binary splits and does not inherently benefit from the gradient of importance values, while KNN's distance metric treats all continuous values uniformly without exploiting their class-association semantics. These properties make both classifiers less responsive to the continuous confidence value structure introduced by RFS-MD compared to gradient-based, ensemble, and deep learning architectures.

Regarding precision and F-score, the scored representation maintains or improves precision values across all classifiers, confirming that the recall gains are not achieved at the cost of increased false positives. F-scores follow accuracy trends closely, reinforcing the consistency of the improvements across the full performance profile. Figures 3 and 4 display the accuracy and recall distributions for all classifiers across 10 folds, illustrating

both the mean performance differences and the reduced variance in the scored condition for several classifiers. Shaded bands showing ± 1 standard deviation around each line.

Table 2. 10-fold cross-validation classification performance: scored (RFS-MD) vs. non-scored (baseline) features.

Classifier	Accuracy (%)			Recall (%)			Precision (%)		F-Score (%)	
	Scored	Baseline	*p*	Scored	Baseline	*p*	Scored	Baseline	Scored	Baseline
DT	95.81 ± 0.54	95.56 ± 0.67	0.1203	96.09 ± 0.71	95.77 ± 0.84	0.1047	95.54	95.36	95.81	95.56
RF	98.24 ± 0.37	97.98 ± 0.49	0.0384 *	98.31 ± 0.46	98.04 ± 0.58	0.0412 *	98.18	97.93	98.24	97.98
KNN	96.18 ± 0.58	95.91 ± 0.74	0.1384	94.21 ± 1.14	93.96 ± 1.32	0.2017	94.41	94.19	94.31	94.07
LR	97.27 ± 0.45	97.05 ± 0.43	0.0393 *	96.89 ± 0.51	96.51 ± 0.47	0.0415 *	97.14	96.82	97.01	96.66
NN	97.79 ± 0.49	97.48 ± 0.61	0.0287 *	98.64 ± 0.47	98.33 ± 0.56	0.0314 *	97.63	97.31	98.13	97.81
CNN	97.72 ± 0.55	97.01 ± 0.58	<0.001 ***	97.82 ± 0.66	96.91 ± 1.22	0.0335 *	97.61	97.12	97.71	97.01

* $p < 0.05$, *** $p < 0.001$.

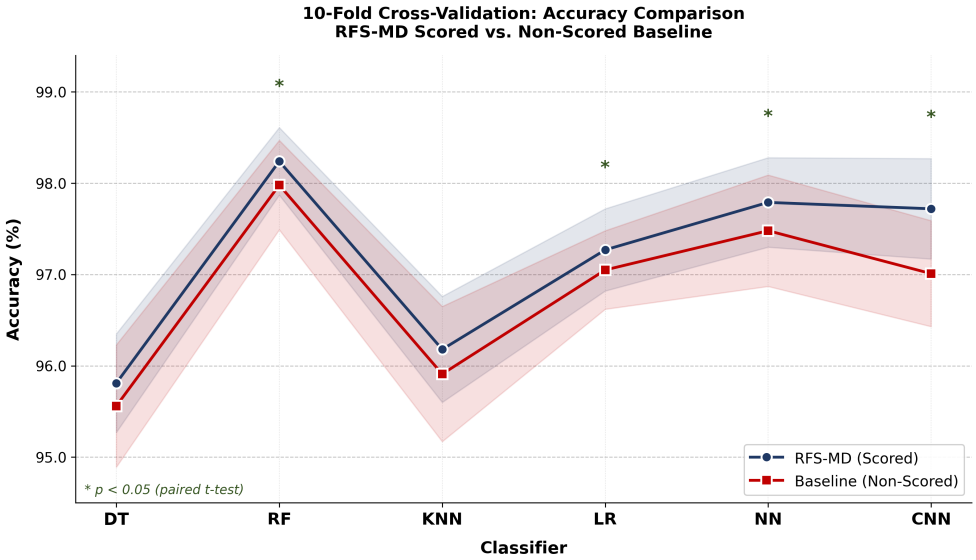


Figure 3. 10-fold cross-validation: accuracy comparison across all classifiers.

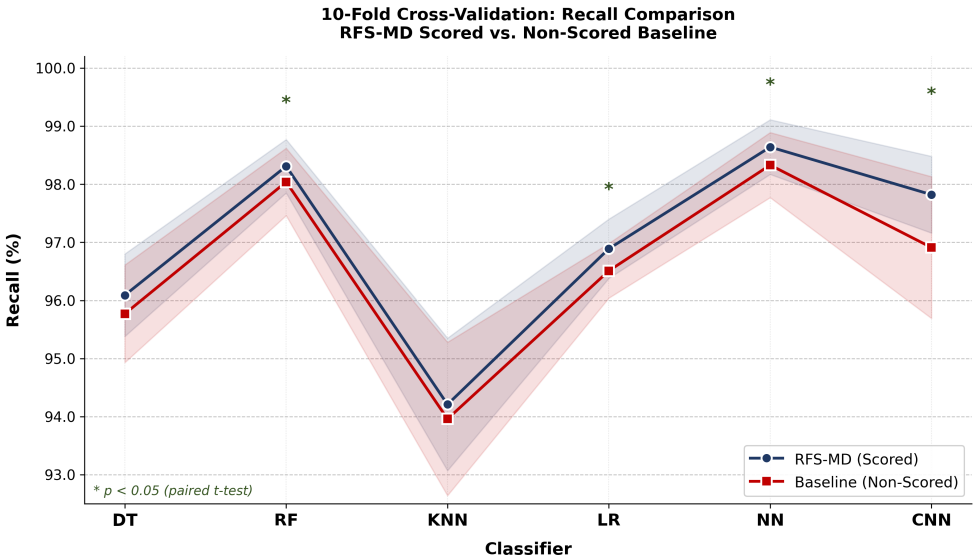


Figure 4. 10-fold cross-validation: recall comparison across all classifiers.

4.1.2. Performance After Hyperparameter Optimization

To ensure the quality, efficacy, and reproducibility of the rule-based feature scoring method, hyperparameter optimization was implemented across all classifiers, tuning the most effective parameters with a wide range of possible values, as shown in Table 1. Table 3 demonstrates the best parameter values for each classifier, while the models' performance under the tuned configurations is illustrated in Table 4.

Table 3. Best hyperparameters obtained from the hyperparameter optimization process.

ML Classifier	Tuned Parameters
Decision Tree	max_depth = None (unlimited depth), min_samples_split = 5, min_samples_leaf = 1
KNN	metric = Manhattan, n_neighbors = 3, weights = distance
Logistic Regression	C = 0.1, class_weight = balanced, penalty = L_2, solver = lbfgs
Random Forest	max_depth = None, max_features = sqrt, min_samples_leaf = 1, min_samples_split = 2, n_estimators = 100
SVM	C = 10, kernel = RBF, gamma = scale
CNN	filters = 64, kernel_size = 3, dropout = 0.3, dense_units = 128, learning_rate = 0.0005, batch_size = 32
Fully Connected NN	batch_size = 64, dropout = 0.1, L_2 regularization = 0.0005, layers = [512, 256], learning_rate = 0.0005

Table 4. Performance of machine learning and deep learning classifiers using tuned hyperparameters on scored and non-scored feature datasets.

Classifier	Dataset	Acc (%)	Prec (%)	Rec (%)	F-Score (%)
DT	Scored Features	95.976	95.729	96.296	96.012
	Non-Scored Features	94.709	94.543	94.963	94.752
KNN	Scored Features	95.753	98.131	93.333	95.672
	Non-Scored Features	95.380	98.116	92.593	95.274
LR	Scored Features	98.361	98.514	98.222	98.368
	Non-Scored Features	97.988	98.214	97.778	97.996
RF	Scored Features	98.584	98.378	98.815	98.596
	Non-Scored Features	98.212	98.655	97.778	98.214
SVM	Scored Features	98.137	98.508	97.778	98.141
	Non-Scored Features	97.690	98.204	97.185	97.692
CNN	Scored Features	97.914	98.356	97.482	97.917
	Non-Scored Features	97.765	98.063	97.482	97.771
FNN	Scored Features	98.361	98.514	98.222	98.368
	Non-Scored Features	97.690	98.348	97.037	97.688

The results show that the rule-based feature scoring demonstrates consistent effectiveness, even under the tuned models. Across all classifiers and optimal model configurations, scored features achieve higher accuracy, recall, and f-score compared with non-scored features, as shown in Figure 5, and achieve higher precision values as well in most classifiers. Our proposed RFS-MD enhances the feature discrimination.

On scored features, LR, RF, SVM, and FNN have accuracy above 98%. This indicates that model tuning increases accuracy in some models, for which the performance behavior remains the same, even with model hyperoptimization on scored and non-scored features, confirming that the improvements are not simply recovered through hyperparameter optimisation; they reflect a genuine enhancement in the quality of the input representation itself. For recall values, it is noticeable that scored features improve or maintain recall across all classifiers, which is a high demand in security applications. In addition, precision values

remain high in traditional ML, while it increases in DL models such as CNN and FNN, indicating that the RFS scoring technique does not increase the number of false alarms.

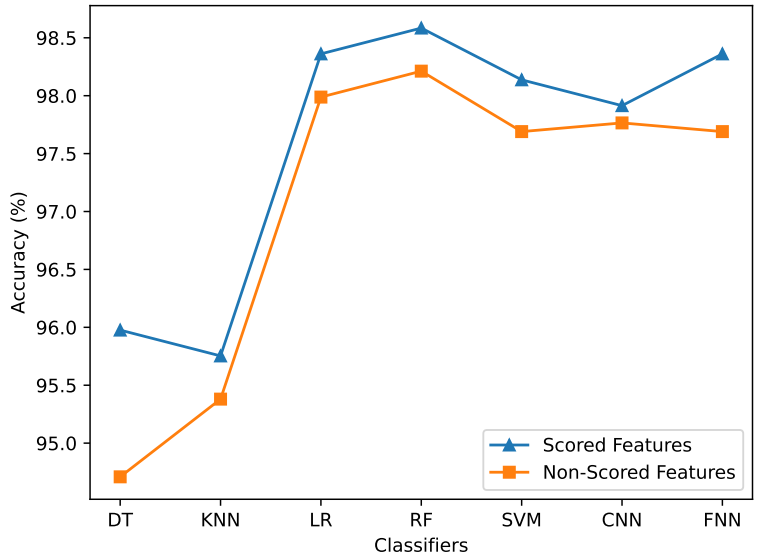


Figure 5. Classifiers' accuracy based on the best parameters.

The chart in Figure 6 shows the gain values of the accuracy and recall across all classifiers. The gain results confirm that the scored representation consistently outperforms the non-scored baseline for all classifiers in accuracy and for six of seven classifiers in recall. This confirms that our proposed RFS technique enhances feature space representation independently of hyperparameter optimization.

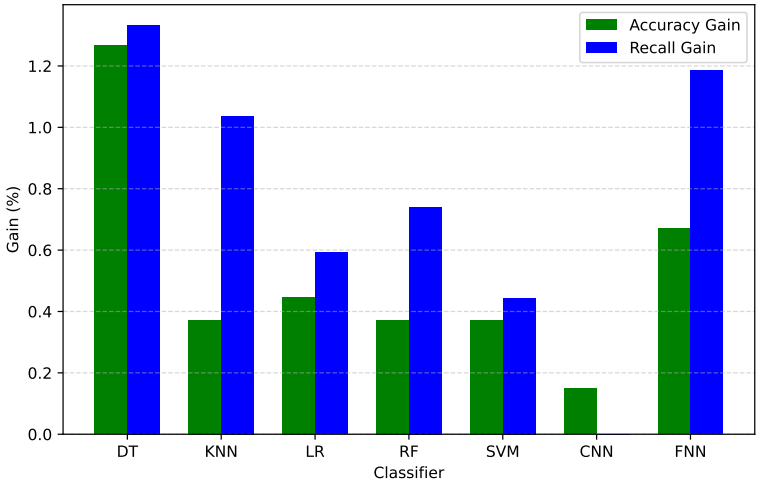


Figure 6. Classifiers' accuracy and recall gain based on the best parameters.

Overall, hyperparameter optimisation reinforces the findings from the cross-validation experiments. Together, the cross-validation and tuned parameter results establish that the rule-based feature scoring mechanism enhances the discriminative quality of the input representation in a way that generalises across model configurations rather than depending on any specific hyperparameter setting.

4.1.3. Comparison with Alternative Feature Weighting Methods

To check whether the performance gains seen with RFS-MD are unique to the rule-based confidence scoring approach or something that simpler weighting methods can also produce, three alternatives were tested alongside it: Chi-Square weighting, Information

Gain weighting, and Random Forest-based Feature Importance. For each method, the 5000 features in the reduced matrix were assigned continuous weights derived from training data, while features that were absent kept a value of zero, matching the same transformation structure used by RFS-MD. All four methods were tested under the same 10-fold cross-validation setup, using the same fold splits and random seed as the main experiments in Section 4.1.1.

Table 5 reports the mean accuracy and recall across 10 folds for all four weighting methods across six classifiers. The results reveal a clear architectural dependency in how the weighting method choice affects performance. For ensemble and distance-based classifiers, RF and KNN, all three univariate methods produce competitive results, with RF-FI achieving comparable accuracy to RFS-MD on RF (97.96% vs 98.24%) and higher accuracy on KNN (97.61% vs 96.18%). This suggests that, for classifiers whose internal mechanisms distribute discriminative weight broadly across features, the choice of continuous scoring method matters less. The most informative contrast emerges for LR and NN. IG and RF-FI reduce LR accuracy to 91.50% and 90.95%, respectively, a degradation of over six percentage points relative to RFS-MD's 97.27%. RF-FI similarly reduces NN accuracy to 93.27%, compared to RFS-MD's 97.79%. This pattern reflects a fundamental property of univariate weighting: scores derived from marginal feature–class associations create a feature geometry that distorts the input space for classifiers sensitive to the relative scaling and interaction structure of their inputs, such as linear and neural models. RFS-MD avoids this distortion because its confidence scores are derived from multi-feature co-occurrence patterns whose consequents are class labels, producing a representation that encodes the interaction structure rather than isolated feature ranks. CNN and DT show more consistent behavior across methods, with RFS-MD providing the highest CNN accuracy (97.72%) among all conditions. Across the full comparison, RFS-MD achieves the best accuracy for RF, LR, NN, and CNN, four of six classifiers, and demonstrates the most consistent cross-classifier performance profile of any method evaluated. The one exception is KNN, for which RF-FI and Chi-Square weighting produce higher accuracy, a finding we report transparently and attribute to KNN's distance-based mechanism, which benefits from the broader importance spread of tree-derived weights in this specific feature space.

Table 5. Feature weighting methods' comparison.

Method	DT Acc/Rec	RF Acc/Rec	KNN Acc/Rec	LR Acc/Rec	NN Acc/Rec	CNN Acc/Rec
Chi-Square	95.74/95.91	97.89/97.95	**97.23**/96.89	96.01/**97.15**	97.21/98.15	97.01/97.31
IG	95.86/**96.22**	97.89/97.95	97.12/**97.04**	91.50/91.68	96.82/**98.67**	97.28/97.60
RF-FI	95.75/96.04	97.96/97.98	**97.61**/**97.87**	90.95/90.37	93.27/97.29	97.09/96.89
RFS-MD	**95.81**/96.09	**98.24**/**98.31**	96.18/94.21	**97.27**/96.89	**97.79**/98.64	**97.72**/**97.82**

All values are percentages. Acc = accuracy; Rec = recall.

4.2. Adversarial Robustness Evaluation

In this subsection, we evaluate the robustness of our proposed RFS-MD model and the baseline model on both RF and FNN, reporting ASR values and recall gain for different feature budgets, with K features attacked by the PGD algorithm for FNN and by query-based attack for RF models. Models in these experiments were trained using standard training without adversarial training to ensure that the effectiveness of model robustness is solely attributed to the proposed RFS feature representation.

4.2.1. Adversarial Attack Evaluation on the Neural Network Model

The ASR performance results for scored and non-scored features with the FNN model are depicted in Figure 7. It can be seen that ASR increases with the number of attacks and that, for the baseline model trained on binary original features, the ASR starts at 5% when only one feature is attacked for each sample and increases as the number of attacked features increases. When modified features reach 50, the ASR value is higher than 20%, and the attack becomes more effective as the perturbation budgets increase. This trend continues with large feature modifications, with about 52% when 125 features were altered. On the other hand, FNN trained with scored features exhibits much greater robustness to the most powerful adversarial examples, such as ASR falling to 2.3% when $k = 1$ (over 50% improvement in our model) and continuously outperforms the baseline model as the perturbation budget increases. ASR with scored features is roughly 34.5% with $k = 100$ compared to 45.4% for the baseline model.

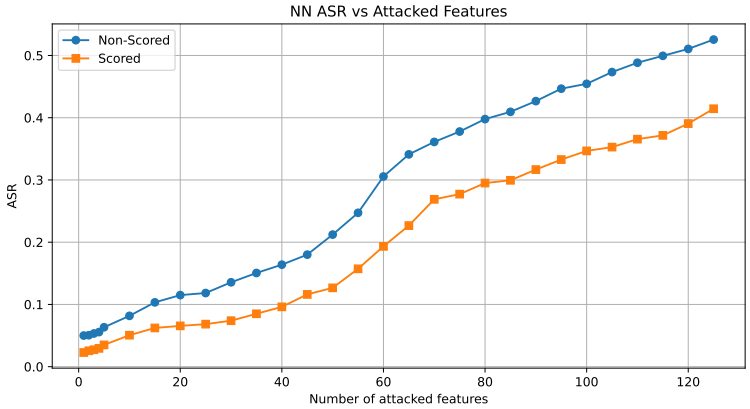


Figure 7. ASR of the FNN model across different k values.

For further evaluation, we computed the recall gain for both tested models; the recall is very important to show how much improvement has occurred in classifying adversarial examples as malware, not benign. The bars in Figure 8 shows that when few features are attacked (k: 1–10), the recall gain ranges from 2.5% to 3%, indicating that our RFS-MD improves adversarial attack resistance even with small feature perturbations. In addition, we noticed that the recall gain increases as the number of features increases linearly. For example, when $k = 10$, the recall gain is approximately 4%, and when $k = 30$, it is approximately 6%. This demonstrates that our proposed model not only has a higher recall than the baseline model but also shows stronger resistance against adversarial attack along a large number of attacked features. Even though the recall gain has a slight fluctuation in more attacks of features, it is still consistently positive at all values of k. The maximum improvement is observed when 115 features were manipulated; the recall gain reached about 12.5%.

Overall, the RFS-MD model-based FNN classifier reduces ASR values across all perturbation budgets and increases recall gain as manipulated features increase, outperforming the baseline FNN classifier, especially over stronger attacks.

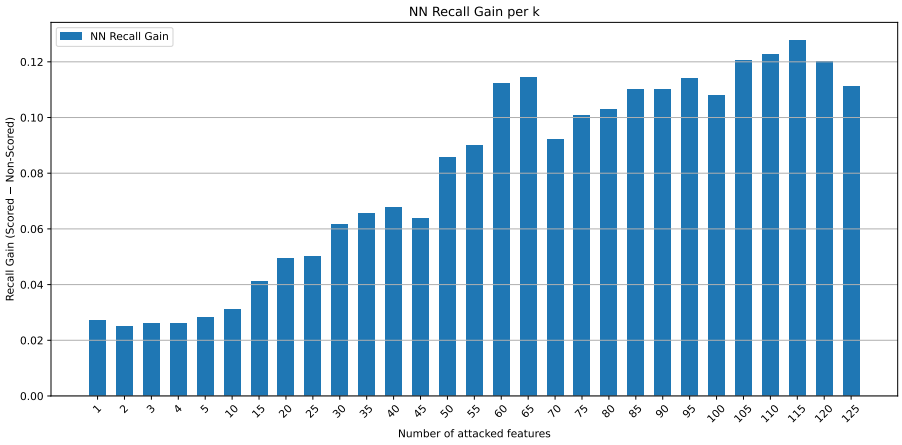


Figure 8. Recall gain of the FNN model across different k values.

4.2.2. Adversarial Attack Evaluation on the Random Forest Model

Our proposed RFS-MD model-based RF classifier is evaluated by ASR and the recall gain against the baseline RF model that was trained on the original binary features. Figure 9 demonstrates that ASR increases as the number of the attacked features increases at a small level of manipulations for non-scored features, starting from 2.5% at $k = 1$, reaching the peak point at around $k = 17$ with ASR of approximately 6.3% for the baseline RF model. This indicates that adversarial attacks can change the decision paths in RF decision trees. A similar pattern is observed for scored features; ASR is lower than the baseline at all adversarial attack levels, in which the ASR reaches about 5.7% at its peak point with $k = 15$, indicating that RFS-MD has a lower peak value that holds maximum ASR, which is also lower than the baseline model.

After the peak point for each model, ASR begins to decrease gradually until reaching 0 value at $k = 47$. This trend indicates that, for a black-box attack on RF, the number of adversarial malware beyond a certain threshold does not improve its resistance performance. Overall, ASR increases linearly with a small number of perturbations and then gradually declines to zero as k increases.

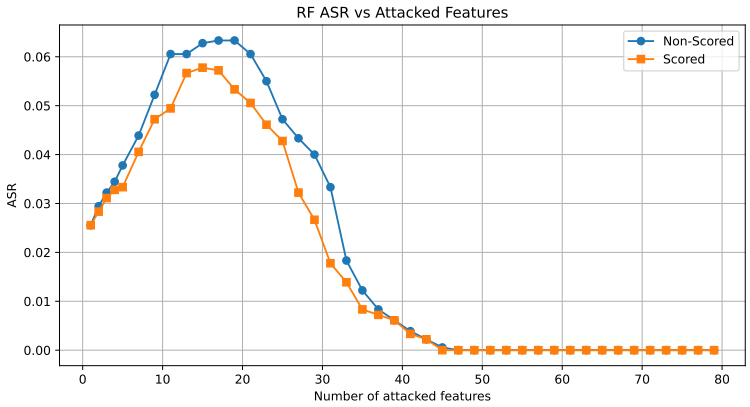


Figure 9. ASR of the RF model across different k values.

The recall gain of the RF model is relatively small, but it remains positive across the most adversarial attack levels, as illustrated in Figure 10. The recall gain is improved from 0 at $k = 1$ to approximately 1.55% as the highest gain at $k = 31$. This demonstrates that the RFS-MD-based RF classifier slightly enhances the adversarial attack detections.

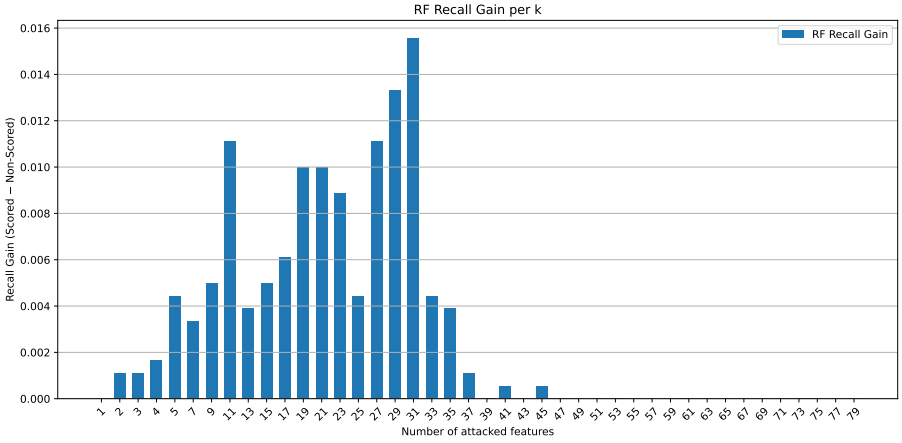


Figure 10. Recall gain of the RF model across different k values.

Our RFS-MD model effectively reduces ASR compared to the baseline model that was trained on the original non-scored features across both classifiers, RF and FNN, while maintaining a positive recall gain, confirming its efficacy in enhancing detection system resistance against adversarial malware apps.

4.3. Model Explainability

4.3.1. Rule-Based Explanations of Model Predictions

To provide human-readable explanations and enhance the explainability of the model's decisions, we used the same CAR rules dataset obtained during the rule generation phase of our methodology, which includes the rules that capture the relationships between the features and the class, and the interactions among the features that contribute to the model's decision. In Table 6, we present some of the model decision explanations for some examples of malware and benign samples, along with their top-5 extracted rules that explain model classification decisions. Each rule includes interacting features in the rule antecedent column, its confidence value reflecting its strength, and its label.

In malware samples, the extracted rules consist of informative interacting features relevant to malicious behavior. We will use Sample 998 from the table as a case study to illustrate the explanatory value of the extracted rules for security analysts. Sample 998 receives a model prediction confidence of 0.95 for the malware class; the top rules we got show that there is a privacy threat to sensitive data and strange network communication behavior. For example, the features $getLine1Number$ and $getSimSerialNumber$ suggest that the app is trying to get unique device identifiers that can be used to track or profile the user. The rule also includes $getSystemService$ with $android.permission.INTERNET$ means that there may be network communication, which could send data back to servers outside the network. Other rules, like $setIcon$ and $setRepeatCount$, tell background or repetitive tasks to hide bad behavior by pretending to be normal. In general, the combination of the privacy data access and transmission operations explains why this sample was classified as malware; these behaviors are strongly associated with data collection and potential exfiltration techniques.

In sample 1815, the highest confidence rules indicate that several malicious tasks are identified through their features; these tasks could be data exfiltration, network monitoring, device fingerprinting, and propagation via shortcuts. The first rule indicates that the presence of $api_call_execSQL$ with the collection of information API using $api_call_getExtraInfo$ supports malware behavior, especially since it has a confidence level of 1. The second rule clarifies that monitoring the changes in WIFI state and holding the permission to access WIFI $(ACCESS_WIFI_STATE)$ simultaneously indicates network

activity tracking, while *onDestroy* suggests hiding tracking or completing the malicious task before destroying it. In addition, malware can cause installing *SHORTCUT* combined with internet access and reading the phone state, as the third rule demonstrated. while *getParams* and *getDeviceId* APIs appear together in the fourth rule, indicating the collection of device parameters and IDs. Rule 5 indicates a possible intent of the hijacking process through the custom *addDataScheme*.

Table 6. Top-5 extracted rules for some malware samples.

Sample Index	Rule Antecedents	Rules Consequent	Confidence
998 prediction (0.95)	api_call_getLine1Number, api_call_obtainMessage	malware	0.947
	api_call_getSimSerialNumber	malware	0.924
	api_call_getDeviceSoftwareVersion, api_call_getSystemService, android.permission.INTERNET	malware	0.919
	api_call_setIcon, api_call_getDeviceId	malware	0.898
	api_call_setRepeatCount	malware	0.898
1815 prediction (0.97)	api_call_getExtraInfo, api_call_setBackgroundResource, api_call_execSQL	malware	1
	intent_WIFI_STATE_CHANGED, permission_ACCESS_WIFI_STATE, api_call_onDestroy	malware	0.958
	permission_INSTALL_SHORTCUT, permission.INTERNET, permission_READ_PHONE_STATE	malware	0.976
	api_call_getParams, api_call_getDeviceId	malware	0.94
	api_call_addDataScheme	malware	0.93
275 prediction (0.98)	api_call_setAnimationStyle, api_call_rotateY, api_call_getLocationOnScreen, api_call_decodeStream	malware	1
	api_call_rotateZ, api_call_onStart, api_call_setHeight, permission.READ_PHONE_STATE	malware	1
	api_call_setLightTouchEnabled, api_call_onTrackballEvent	malware	0.996
	api_call_getSubscriberId	malware	0.958
	api_call_writeByte, api_call_setWebViewClient	malware	0.94

According to the extracted rules of sample 275, this sample attempts to reveal private information and sensitive data from the device while hiding its malicious activities behind normal app operations; the confidence value of 1 for rules 1 and 2 indicates the semantic strength of these combined features, supporting the idea of applying these operations to hide the malicious main tasks, like animations, rotations, and touch handling. *getSubscriberId* and *READ_PHONE_STATE* APIs suggest that the app

could track data and access sensitive device identifiers, while *api_call_writeByte* and *api_call_setWebViewClient* demonstrate data processing and possible web communication. Overall, these malware rules indicate that the application uses obfuscation techniques and privacy data collection, which explains why the detection model classifies it as malware.

Benign samples tell a different story, as shown in Table 7. Sample 8595 received a confidence score of 0.98, among the highest in the evaluated set. Its top explanation rules cover HTTP request setup, browser cache paths, external storage, file handling, cookie access, and screen UI calls. None of these is exotic. They show up regularly in browsers, file managers, and apps that load web content. What matters here is not any single API but the full picture: these features cluster together in a way that matches benign training patterns closely, and none of the high-confidence malware rules fire. The 0.98 score reflects both factors working together.

Table 7. Top-5 extracted rules for some benign samples.

Sample Index	Rule Antecedents	Rules Consequent	Confidence
8595 prediction (0.98)	api_call_addRequestHeader, api_call_setAppCachePath	benign	0.976
	api_call_getExternalStoragePublicDirectory, api_call_getDataString, api_call_isFinishing	benign	0.92
	api_call_guessFileName	benign	0.9
	api_call_getCookie	benign	0.85
	api_call_setSystemUiVisibility	benign	0.87
7773 prediction (0.59)	api_call_getPreferences, api_call_canGoBack, api_call_requestFocus	benign	0.94
	api_call_onReachedMaxAppCacheSize, api_call_getDefaultSize, api_call_onTouch	benign	0.85
	api_call_setDisplayHomeAsUpEnabled, api_call_getChildAt, api_call_putBoolean	benign	0.84
	api_call_setSupportedAdSizes, api_call_getLocationOnScreen	benign	0.79
	api_call_setCustomClose, api_call_setOnTouchListener	benign	0.78

Sample 7773 sits at 0.59, noticeably lower. Its rules point to navigation calls (canGoBack, requestFocus), stored preferences, ad display logic, and touch handling. None raises obvious red flags on their own. But the confidence values drop from 0.940 on the first rule down to 0.780 by the fifth. In the malware explanations, that same range stays between 0.940 and 1.000. The narrower spread here means the model found a benign match, but not a strong one; the associations exist in the data, but they are not dominant.

It is important to note that this interpretation is explicitly probabilistic and context-dependent, not an absolute semantic assertion about any individual API. getCookie, WebView calls, and UI controls show up mostly in benign apps in this dataset, which is what the confidence values capture. The same calls can and do appear in malicious apps, though, when they sit alongside aggressive permission requests or hidden network activity. The RXAI explanation does not claim that these APIs are inherently safe; it reports the empirical co-occurrence pattern observed across the training corpus. A security analyst interpreting these rules should read them as: the feature combinations present in this sample are statistically associated with benign behavior in the training data, with the stated probability, not that these features cannot possibly indicate malicious intent in other contexts.

4.3.2. Fidelity Evaluation of Rule-Based Explanations

To evaluate the effectiveness of the decision explanations of our proposed RXAI model, a subset of 200 samples for fidelity evaluation was drawn from the held-out test partition using random stratified sampling, with class balance maintained at 100 samples per class, ensuring the selection process was unbiased and representative of the class distribution in the full dataset. The distribution of fidelity scores is shown in the violin plot (Figure 11), which provides a comparison between fidelity score distributions for both malware and benign classes. Table 8 presents the Fidelity statistics for malware and benign samples.

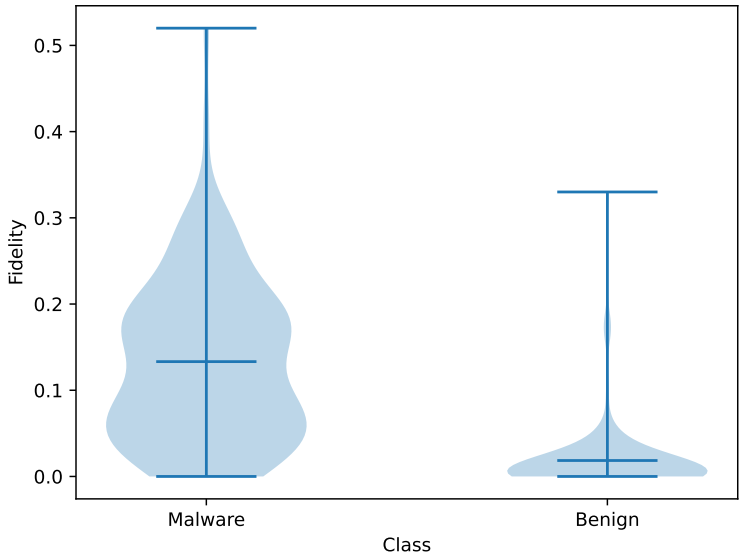


Figure 11. Fidelity scores distributions for malware and benign classes.

Table 8. Fidelity statistics of Malware and benign samples.

Samples	Mean	Median	Standard Deviation	Maximum
Malware	0.133	0.12	0.096	0.52
Benign	0.0185	0.01	0.045	0.33

For the malware sample, we have wide distributions for fidelity score, which mean that, for a specific fidelity value, we have a density of malware samples. The mean value is 0.133, with a median of 0.12 and a standard deviation of 0.096. Scores ranging from 0 to 0.52 with density area, indicating that most of the malware samples have observed changes in prediction values. In addition, almost half of the malware samples extend towards higher scores, making a longer upper tail. This violin shape demonstrates the influence of the top rules across different samples in the model explainability process. In contrast, the benign samples have lower fidelity scores and a small density area that is located in the low fidelity values. This indicates that a small number of benign samples have a fidelity of more than 0, with a mean of 0.0185. Samples with the highest scores are rare, with scores up to 0.33. The overall median is 0.01, and the standard deviation is 0.045.

The difference in mean fidelity scores between malware (0.133) and benign (0.0185) indicates the different influence of removing the features of the sample top rules on the two classes, in which the shape of the fidelity score distribution and the spread confirm this distinction. From a global perspective, the extracted rules collectively capture the model behavior more effectively for malware samples than for benign samples.

A Mann–Whitney U test conducted on the two fidelity score distributions yields $U = 9186.5$, $p = 2.53 \times 10^{-25}$, and a rank-biserial effect size of $r = 0.919$. An effect

size of $r = 0.919$ indicates that in 91.9% of all possible malware–benign sample pairings, the malware sample has the higher fidelity score. This is an extremely large effect by any standard interpretation. The statistical evidence here is strong enough to rule out chance as an explanation for the fidelity difference. The finding that RXAI explanation rules carry substantially more weight in the model's decisions for malware samples than for benign ones is not just an observed trend but a result that holds up under formal statistical testing. Also, these numbers mean the fidelity difference between classes is not a marginal signal that depends on a large sample size to reach significance; it is a structurally dominant pattern that would remain statistically detectable with considerably fewer than 200 observations. The 200-sample evaluation, therefore, does not limit the conclusions that can be drawn; rather, the strength of the effect confirms that the finding is stable and would generalize to the broader dataset.

4.4. Temporal Generalization Evaluation

To evaluate whether the RFS-MD framework generalizes beyond the time period covered by the primary training data, a temporal evaluation was performed using a time-based partition of the dataset. The most recently dated 20% of the existing 9000-sample corpus, approximately 1800 samples, formed the held-out temporal test partition. The remaining 80% constituted the temporal training set, ensuring that no test sample shares a collection period with any training sample. A critical methodological distinction was enforced in this evaluation between temporal drift and data leakage. To prevent data leakage in the temporal context, all preprocessing operations, MI-based feature selection, FP-Growth rule mining, and RFS feature scoring, were computed exclusively on the temporal training partition and applied to the test partition using pre-computed parameters, with no access to test-period feature distributions or labels at any stage. This protocol represents a deliberate methodological improvement over the primary random-split evaluation. It should also be noted that a family-disjoint split was not feasible for this study, as the dataset does not include malware family classifications, a limitation acknowledged in Section 5.4. Table 9 reports classification performance under temporal conditions. Absolute performance drops moderately across all classifiers, ranging from approximately 1.5 to 5.8 percentage points in accuracy and reflecting the natural behavioral divergence of newer malware from training-period patterns. This drop is expected and reported transparently. The RFS-MD representation maintains its accuracy advantage over the non-scored baseline for six out of seven classifiers. For recall, four classifiers show consistent improvement while Decision Tree, KNN, and Logistic Regression show marginal negative or near-zero differences.

Table 9. Temporal generalization evaluation.

Classifier	RFS-MD Acc (%)	Baseline Acc (%)	RFS-MD Recall (%)	Baseline Recall (%)
DT	91.17	91.44	91.78	92.11
RF	94.83	92.89	95.56	93.44
SVM	93.72	92.11	94.22	92.78
KNN	92.06	90.28	91.33	91.56
LR	93.06	92.56	93.44	93.78
FNN	95.17	93.11	96.44	94.33
CNN	95.56	93.44	96.83	94.67

5. Discussion

5.1. Malware Detection Performance

Cross-validation across 10 folds showed that rule-based feature scoring produced statistically significant gains for four of the six classifiers tested, CNN, NN, RF, and LR, across

both accuracy and recall. All six classifiers show a positive directional recall improvement under the scored representation. These significant gains held steady across folds with low variance, which rules out the possibility that they were a product of how the data happened to be split. From a security angle, the recall results matter most. Swapping binary feature values for rule-based confidence scores made these models noticeably better at catching malicious patterns, cutting down on false negatives across classifier types that respond well to continuous feature values. Precision held up or improved at the same time, so the sensitivity gains did not come with a spike in false alarms.

How much each classifier improved depended on its architecture. CNN picked up the biggest statistically significant gain, with accuracy at $p < 0.001$ and recall at $p = 0.0335$, and its recall variance dropped too under the scored condition. That suggests that the convolutional structure is especially good at making use of the continuous representation that RFS-MD introduces. NN and RF both improved at a moderate but significant level. LR showed that even a simple linear boundary gets something out of the richer input geometry that confidence scores provide. DT and KNN moved in the right direction but not by enough to reach significance. That fits with how they work: tree splits and nearest-neighbor voting are just less responsive to the gradient in confidence values than gradient-based, kernel, or ensemble approaches are. Taken together, these results show that the benefit of RFS-MD scoring is tied to classifier architecture, and the clearest gains show up in models built to take advantage of continuous feature spaces. Section 4.2 then shows what each classifier can do at its best, with tuned hyperparameters applied on top of the scored representation.

The consistent improvements across architecturally diverse classifiers, alongside statistically validated gains for four of six classifiers under cross-validation, demonstrate the resilience and generalizability of the proposed approach. Our RFS-MD captures interactions between features and their relationships with class labels, and highlights the impact of integrating the association rules into the feature representation space. Table 10 presents a comparison between our proposed RFS-MD approach and some previous related studies. The RFS-MD approach shows a unique and comprehensive approach, maintaining a clear advantage in explainability and robustness. Previous studies [48,52] have enhanced feature representations based on graph learning or multimodal fusion, obtaining high accuracy and recall through the use of complex models. While other studies implemented hybrid feature extraction and ensemble or deep learning models to improve classification results over multiple Android datasets [53,54]. These methods show that they can make good predictions but depend on either expensive models or complex feature selection and scoring methods. They do not focus on making the results understandable or maintaining high performance gains across different classifiers.

In contrast, our proposed RFS-MD approach follows a different approach by putting rule-based feature scoring directly into the input feature representation. It uses association rules to highlight the interacting features and their relationships with the class label, adding feature scores and making the feature space expressive and understandable. Unlike traditional feature selection methods, such as Wei et al. and Pathak et al. [55,56], which focus on reducing dimensionality or weighting features based on statistical importance. RFS-MD has the same or better accuracy, up to 98.584%, and always improves recall in all classifiers, with the best case being 98.815%, while maintaining precision levels in the remaining cases. Improvements in recall are very important; many studies either do not report recall directly or get high accuracy without making sure they can find malware samples. In addition, studies such as [57,58] employ multimodel fusion or redundancy reduction to enhance model performance without improving the recall values or increasing model resistance against adversarial samples.

Overall, the comparisons indicate that other studies work on enhancing malware detection performance or designing feature engineering techniques. While our RFS-MD approach presents more balance and an effective strategy, integrating rule-based feature scoring into the feature representation and uses the extracted rules to provide more explainable decisions. The RFS-MD approach bridges the gaps between performance, robustness, and explainability within a unified framework, addressing important limitations in current malware detection frameworks and providing a deployable and reliable adversarial malware detection system.

Table 10. Comparison with recent malware detection studies.

Study	Dataset	Feature Type	Feature Processing	Model	Best Acc (%)	Recall (%)	Key Contribution
[48]	MalNet-Tiny	Graph-based (FCG)	Interpretability-driven FS (IG, GradCAM, SHAP)	JK-GraphSAGE (GNN)	94.47	94	Representation-centric approach improving GNN performance via explainable feature engineering
[53]	124K Android apps	Static (API calls, opcodes) + Dynamic (network traffic)	Feature-based ML (MI, PCC, T-Test, SFS, PCA, Chi-square)	Ensemble ML	0.973	–	Demonstrated effectiveness of feature-based ML with ensemble models along with API calls and opcodes
[55]	AndroZoo	Static (API, Permissions, Opcodes)	Dynamic Weighted Feature Selection (DWFS) using RF importance	GNN	95.56	95.68	Robust detection under obfuscation using weighted feature importance
[52]	Drebin	Static + Dynamic + Graph	Graph augmentation + multimodal fusion	Multimodal DL	99	99	Graph-based multimodal learning for robust malware detection
[57]	Multiple datasets	Hybrid (Static + Dynamic)	Multimodal deep fusion	CNN/GNN Ensemble, DenseNet-GIN	90.6	91.1	Multimodal feature integration to improve classification
[54]	Drebin, AndroZoo, VirusShare	Static (API, Permissions, Intents)	Information Gain + hybrid preprocessing	DNN, ML models	93	93	Hybrid framework improving feature quality across malware families
[56]	Defense Droid	Static (Permissions)	Feature importance (GB)	RF, KNN, NB, DT	93.96	91.54	Reduced dimensionality while maintaining performance
[58]	Drebin, CICAndMal2017	API, Permissions, Opcodes	RRFS	SVM, RF, ML models	98.5	91.39	Dimensionality reduction with high accuracy
RFS-MD (Proposed)	AndroZoo	API, Permissions, Intents	MI + Rule-based Scoring	RF, FNN, ML/DL	98.584	98.815	Rule-based feature scoring improving performance, robustness, and explainability

5.2. Model Robustness: Adversarial Attacks

The results of adversarial attacks implemented show strong evidence that the rule-based features scoring technique effectively enhances the robustness of the malware detection models under both white-box and black-box feature attacks. The consistent trend across all experiments shows that models trained on scored features exhibit lower sensitivity against adversarial perturbations and preserve a higher malware detection performance compared to the baseline model trained on the same non-scored features. This is shown by ASR values results of the implemented models and by the recall gain, which presents the increase in the RFS-MD model recall values over the baseline model recall values on the same number of attacked features.

In the white-box attack, a PGD attack was applied on FNN models, in which the ASR values increased sharply as the number of attacked features increased, reflecting the expected behavior of the PGD attacks. However, the degradations in the performance of the model trained on scored features are less severe than the decline in the performance of the model trained on non-scored features; it has higher ASR values. This is because the rule-based feature scoring technique scores informative features based on high-confidence extracted matching rules, which prioritizes the effective features that are strongly associated with discriminative behaviors of a specific class. As a result, the implemented feature-based attack on less important features has less influence on model decisions, even when it attacks the strongest correlated features; their stronger associations with class make it difficult for the PGD attack to change model decisions.

The positive results of the recall gain of the proposed RFS-MD model indicate that it can correctly identify malware samples even when the number of attacked features increases. This is particularly important in the cybersecurity field, where false negatives lead to a critical risk. In general, the results of recall gain demonstrate that RFS-MD not only enhances classification performance but also improves adversarial attack robustness of malware detection models, making the model more reliable in real-world malware detection scenarios.

In a black-box attack, the robustness of the RF is significantly improved, unlike FNN, which depends on gradients, because the RF uses discrete decision boundaries with the ensemble method, which makes it more resistant against small feature modifications. RF based on the RFS-MD model enhances robustness over a wide range of attack strength levels with little impact on recall. This higher performance of the RF model is due to the combination of the most important interacting features with ensemble decision-making, for which the critical features control the splitting points of the decision trees, making it challenging for adversarial feature manipulation to change multiple decision paths simultaneously. In a black-box attack, in which attackers do not know the model structure, this impact is further enlarged, and the effectiveness of the attack is reduced.

The findings demonstrate that using the RFS technique improves malware detection performance and increases robustness against adversarial attacks, in which the feature importance scores derived from the relevant rules are integrated into the learning process. This makes the model an effective defense mechanism against different types of adversarial attacks in cybersecurity systems.

To further analyze the robustness limits of the proposed framework, we consider a stronger threat model involving an adaptive attacker. The rule dataset, which is used for the feature scoring process, is maintained as a separate and protected component of the deployed trained model. This separation limits direct access to the rule base, reducing the feasibility of white-box attacks. In addition, some security mechanisms could be adopted, such as access control, model–rule decoupling, and secure storage of the rule. Even with partial knowledge of the scoring mechanism, generating adversarial examples remains

challenging due to the nature of feature interactions and the multiple rules of high relational features. As a result, the adaptive attacker could attempt to avoid the strongest features, but the structural constraints imposed by the rules inherently increase the difficulty of effective evasion.

One worthwhile direction for future work is validating adversarial robustness at the APK level. Building on the design-based functional integrity established in this study, recompiling the modified samples and executing them in a dynamic sandbox would confirm that the inserted benign features are preserved correctly after repackaging and that the malware's payload remains active.

5.3. Explainability

Our rule-based explanation method for the malware detection model relies on the relationships among the strongest features of the rules and identifies how much these interacting features push the model to make a decision for one of the class labels. Choosing the top confident rules makes model decisions more understandable by helping malware analysts or security users understand which features could lead to more malicious behaviors, construct a clear idea about complex attack behavior, and identify the malware security-based target. Unlike methods that depend only on important features, which consider isolated indicators in model decisions, making it hard to capture the big picture of the malware target. Consequently, the decision explanations produced by RXAI, as we saw in the explanation results of the malware and benign predicted samples, go beyond conventional feature interpretability. The combinations of features associated with the classification output enable the model to provide actionable insights that can help security analysts understand, verify, and interact with detected threats. In addition to these advantages, it also reinforces trust between users and the security system, especially in cybersecurity fields, where model decisions are no longer treated as a black box but as logical conclusions derived from informative patterns. As a result, the RXAI approach not only works as an explainability model but also works as a bridge between automated malware detection and security analysis from a human perspective.

The fidelity results show that the proposed RXAI is aligned with the behavior of the model decisions, particularly for malware samples. This indicates that the results' explanations are not only approximate or heuristic but also refer to the designed prediction model. This alignment is necessary to ensure the reliability and validation of the model's explainability and provide actionable insight, especially in cybersecurity applications. On the other hand, when we have high fidelity results for the malware samples, it means that the top selected explanation rules carry malicious feature relationships that associate the rule's features together, at the same time, associate these interactive features with the class label. The large number of malware samples that have high fidelity demonstrates that rule-based explanations have successfully extracted the interacting features through the top confident rules. While benign samples have fewer samples that have high fidelity with a lower fidelity average than the malware samples, supporting the non-discriminative nature of benign features, which is naturally more difficult to capture using compact rule sets. These findings support the reliability and explainability of our proposed RXAI model for security systems.

Table 11 demonstrates the comparisons between our proposed RXAI and other studies that adopted diverse explainability XAI methods. The most widely used XAI methods are SHAP and LIME, which are considered post hoc, model-agnostic techniques that provide feature importance scores for specific model predictions and use surrogate models without modifying the underlying model [59,60]. LIME works as a local explanation method, while SHAP may work as a local and global method. In more advanced model-specific approach,

such as [48], gradient-based methods are used where researchers combined Grad-CAM and SHAP to produce post hoc, local explanations within deep learning architectures. In addition, the attention mechanism as a model-specific was adopted in [61], where it provides intrinsic explanations by inserting the attention weights directly into the CNN model, providing local and global explanations. Feature importance with memory-based reasoning employed is in [62], providing local and global explanations, while [63] adopted graph neural networks (GNNs) with CFGs control flow graphs (CFGs), identifying important subgraphs that provide model-specific post hoc local explanations.

Across all of these studies, feature importance and attention weights or structural attribution were used to provide explanations of the model decisions, without explicitly representing feature interactions in rule form. In contrast, rule-based explanations are used in [64], where fuzzy decision trees were extracted before model implementation, which is limited to intrinsic explanations with no feature importance and does not guarantee alignment with model decisions. Although these model explanation methods provide useful information to humans, they remain feature-based analyses with no explicit strong relationships among the related features. As a result, they offer a decision process with a fragmented view, which depends only on the independent feature contributions. Additionally, robustness considerations through adversarial attacks are absent from these studies, indicating that they do not provide behavior patterns related to malicious functionalities after feature modifications.

Our RXAI considers a comprehensive explainability model, as the table demonstrates, compared to other XAI techniques, providing a complete picture of the model's decision behaviors. It supports both intrinsic and post hoc explainability; rule-based feature scoring produces new feature representations into the model input, while extracting the top confident matching rules provides explanations after model predictions. Furthermore, the proposed RXAI approach provides local and global explainability by identifying feature relationships derived from the association rules. At the global level, classification association rules (CARs) are extracted from the entire dataset, which preserve globally meaningful feature combinations and provide semantic explanations for general model decisions. At the local level, RXAI identifies the group of rules that match specific sample features to provide a prediction explanation for the sample level. This dual capability allows the model to bridge the gap between the global understanding of model results and the local predictions' transparency, which is not available with LIME or SHAP alone. The proposed RFS-MD approach incorporates robustness against adversarial attacks by highlighting features that remain discriminative and informative after the attack process, which is reflected in inclusion evaluation metrics such as fidelity and ASR. Consequently, it unifies explainability, reliability, and robustness within a single malware detection framework, making it suitable for XAI cybersecurity applications.

Table 11. Comparison of explainability methods for Android malware detection.

Study	XAI Method	Explanation Type	Provides Rules	Feature Importance	Model Explanation	Robustness Awareness	Evaluation Metrics
[59]	LIME (hierarchical)	Local (post hoc, model-agnostic)	×	✓	✓	×	SHAP values
[60]	SHAP + feature analysis	Local + Global (post hoc)	×	✓	✓	×	Accuracy, SHAP consistency
[48]	GradCAM + SHAP	Local (post hoc)	×	✓	✓	×	Accuracy, explanation quality
[61]	Attention mechanism	Local + Global (intrinsic)	×	✓	✓	×	Accuracy, attention visualization

Table 11. *Cont.*

Study	XAI Method	Explanation Type	Provides Rules	Feature Importance	Model Explanation	Robustness Awareness	Evaluation Metrics
[62]	Feature importance + memory-based XAI	Local + Global	×	✓	✓	×	Accuracy, robustness analysis
[63]	GNN matching subgraph	Local (post hoc)	×	✓	✓	×	Fidelity, subgraph similarity
[64]	Fuzzy rules / Decision tree	Local (intrinsic)	✓	✓	×	×	Accuracy
RFS-MD (Proposed)	**Rule-based (RXAI)**	**Local + Global (intrinsic + post hoc)**	✓	✓	✓	✓	**Fidelity, Accuracy, Recall, ASR**

5.4. Methodological Considerations and Limitations

Concept drift represents a structural challenge for any detection system trained on historical data: as the malware threat landscape evolves, newly emerging malware families may exhibit feature patterns not represented in the training corpus, and both the trained classifiers and the CAR rule dataset may become less effective over time. This limitation is shared by all ML and DL-based malware detection systems and is not unique to the proposed framework.

The labeling strategy used in this study was designed to establish a high-confidence, reproducible baseline, which requires more than 30 VirusTotal engine detections for malware samples and zero detections for benign samples. This strategy prioritizes label reliability over distributional completeness: every included malware sample carries a strong multi-engine consensus, and the clean negative class provides an unambiguous contrast. The trade-off is that the dataset excludes the most challenging cases for operational detection: grayware applications with limited but real malicious functionality, newly emerging malware families with detection counts below 30, and zero-day threats not yet indexed by commercial engines. Future work will specifically address this gap by extending the RFS-MD framework to low-consensus detection scenarios. The planned approach is to use the high-confidence CAR rule base established in this study as a reference anchor, then evaluate how rule-matching patterns change as the VirusTotal threshold is progressively lowered from 30 to 10 and then to 1–5 detections (the grayware range). This staged approach allows the framework to leverage the stable, well-characterized rule patterns from confirmed malware to guide the detection of borderline samples, rather than attempting to learn from ambiguous labels directly.

A key methodological consideration in this study is the use of a perfectly balanced training distribution (4500 malware vs. 4500 benign samples). Although operational Android malware detection typically encounters highly imbalanced class distributions, the balanced design was a deliberate choice driven by the requirements of the FP-Growth rule mining phase. Under imbalanced conditions, association rule discovery is biased toward majority-class patterns, reducing both the coverage and the confidence reliability of minority-class rules. This would have directly weakened the RXAI explainability component, which depends on a rich and representative CAR dataset for both classes. The balanced design also separates the contribution of the RFS-MD feature transformation from any confounding effects tied to class distribution, allowing the scored versus non-scored comparison to be read as a direct test of what the feature representation itself brings to the table. Extending the framework to handle real-world imbalanced distributions through cost-sensitive learning is identified as an important direction for future research

The absence of malware family labels in the dataset is consistent with the binary detection focus of the proposed framework. Family-level classification is a separate task that falls outside the primary scope of RFS-MD and is identified as a direction for future work. Extension to datasets with obfuscated or polymorphic malware family annotations remains an important future research direction.

A methodological consideration specific to rule-based feature scoring concerns whether class-derived confidence values introduce label-specific bias into the input representation, potentially creating overfitting or optimistic evaluation. Three structural properties of the RFS-MD pipeline mitigate this risk. First, all CARs are generated exclusively from the training partition, and the test partition is transformed using a fixed pre-computed mapping derived entirely from training data, as described in Section 3.5.3, ensuring that no test-set label information enters the scoring process at any stage. Second, the minimum support threshold (minsup = 0.01) and confidence threshold (0.7) act as regularization constraints on the rule base, filtering out rare or statistically weak associations that are more likely to represent training-specific noise. Third, the 10-fold cross-validation results in Section 4.1.1 provide direct empirical evidence against overfitting: if the scoring created a representation overfitted to one training partition, performance would vary substantially across folds. The consistently low standard deviations observed across all classifiers, particularly for classifiers that benefit most from the scored representation, such as CNN and NN, confirm that the improvements generalize across different data partitions rather than reflecting training-set memorization.

This study involved some other limitations that should be highlighted. First, one natural direction for extending this work is bringing dynamic behavioral features into the RFS-MD framework, things like system call sequences, network traffic patterns, and runtime permission requests. The rule-based scoring mechanism works on binary feature vectors, so it can, in principle, handle any feature type that can be represented in that format. Adding dynamic features would make the framework harder to fool through obfuscation-based evasion strategies that manage to hide malicious behavior during static analysis, and would give both the detection and explainability components a richer picture of what an application is actually doing at runtime.

5.5. Cross-Dataset and Temporal Generalization

The temporal evaluation in Section 4.4 reveals a nuanced picture of RFS-MD generalization. Six classifiers maintain their accuracy advantage under temporal conditions, and deep learning models (NN, CNN) show consistent recall improvements of over two percentage points. Decision Tree, KNN recall, and Logistic Regression recall show marginal negative results under temporal shift; a pattern that reinforces the interpretation that the rule-based scoring benefit is most pronounced for classifiers designed to exploit continuous feature representations. The consistency of the RFS-MD advantage across both the primary random-split evaluation and the more demanding temporal evaluation provides evidence that the rule-based feature transformation captures behavioral co-occurrence patterns that are not merely artifacts of a specific collection period. Classification Association Rules derived from training-period data continue to provide discriminative information about temporal test samples, suggesting that the fundamental feature interaction patterns that distinguish malware from benign applications exhibit some stability over the temporal range covered by this dataset.

Two methodological distinctions are relevant here. Temporal drift, which is the natural evolution of the malware threat landscape, is an inherent challenge for any static model and is not a limitation specific to RFS-MD. The temporal evaluation shows that the scored representation retains its detection advantage over the binary baseline despite the performance

reduction caused by temporal drift. Data leakage, the methodological risk of test-period information entering preprocessing, was explicitly mitigated in the temporal evaluation by fitting all preprocessing steps on the temporal training partition only. This distinction, and the strict protocol used to address both concerns separately, represents a methodological improvement that strengthens the validity of our generalization assessment.

Cross-dataset evaluation on independent benchmarks such as CIC-AndMal2017 remains an important future direction. Full cross-dataset validation would require feature re-extraction using the same Androguard pipeline, vocabulary alignment verification, and either transfer of the pre-trained RFS-MD pipeline or retraining on the target corpus. Broader cross-dataset evaluation is left as a formally scoped future direction.

5.6. Feature Weighting Strategy and Framework Positioning

The comparative evaluation of feature weighting methods in Section 4.1.3 provides important context for understanding where RFS-MD sits relative to established alternatives. Classical statistical measures, Chi-Square, Information Gain, and RF-FI, are computationally efficient and effective at identifying individually informative features. However, the comparison reveals a structural limitation that becomes consequential in the malware detection context: these methods assign importance scores based on the marginal statistical relationship between each feature and the class label in isolation, without modeling the logical co-occurrence dependencies between features that define malicious behavioral patterns in Android applications. Modern obfuscated malware rarely relies on a single suspicious indicator. Its malicious profile emerges from the simultaneous presence of specific feature combinations, a particular permission granted alongside specific API calls in the context of specific intent declarations, where the combination is diagnostic even when no individual element is uniquely suspicious. Chi-Square, IG, and RF-FI score each feature independently and, therefore, cannot encode this interaction structure into the representation. RFS-MD overcomes this limitation by grounding its scores in classification association rule confidence values, which capture exactly these multi-feature co-occurrence patterns. The result is a feature representation that reflects the combinatorial logic of malicious behavior rather than a ranking of isolated feature relevance, producing more consistent detection performance across classifier architectures and greater stability under conditions where the marginal feature signal is weakened by obfuscation.

A related advantage concerns explainability. SHAP-based feature weighting, a more sophisticated alternative not evaluated here due to the computational cost of fold-level SHAP computation at this dataset scale, shares the same fundamental limitation: it identifies which features are important for a given prediction but cannot express the logical dependence between features. A SHAP explanation states that feature X contributed positively to the malware prediction; an RFS-MD explanation states that features X, Y, and Z co-occur with malware in 91% of training samples matching this pattern. The second form of explanation is directly actionable for a security analyst; it describes a behavioral signature rather than a numerical attribution. This is the core of the Trustworthy AI argument for RFS-MD: the explanations are grounded in logical co-occurrence relationships that are inherently human-interpretable, rather than being post hoc approximations of the model's decision logic derived from a separate explanation process.

Adversarial training was deliberately excluded from this comparison. As discussed in Section 4.2, the adversarial robustness experiments in this study assess the framework's inherent structural resistance to feature-level perturbations, not the effect of training against them, a methodologically distinct question reserved for future work. SHAP-based weighting comparison and adversarial training baselines are both identified as important extensions of the current evaluation framework

6. Conclusions

This study presented a novel rule-based feature scoring framework for malware detection (RFS-MD), functioning as an explainable adversarial defence system. We leveraged the FP-Growth algorithm to extract high-confidence classification association rules (CARs); the proposed RFS-MD uses the selected CAR rules to produce scored feature representations as input to ML and DL classifiers and to provide a rule-based explanability technique (RXAI) model decisions. These integrations enable informative feature representations for classification models by capturing both the meaning of individual features and their underlying relationships. Several experiments implemented the RFS-MD approach compared to the baseline non-scored features representation under multiple ML and DL classifiers, such as DT, KNN, LR, RF, SVM, CNN, and FNN, to assess the proposed RFS-MD approach. Multiple evaluation metrics based on experiment type were adopted in this study. We used accuracy, recall, precision, f-score, ASR, the gain in recall and accuracy, and fidelity.

The proposed RFS-MD approach demonstrates positive recall improvements across all six classifiers under 10-fold cross-validation, with statistically significant gains confirmed in four: CNN, NN, RF, and LR, for which CNN reaches $p < 0.001$ for accuracy and $p = 0.0335$ for recall, confirming that the observed gains are not attributable to a favorable data partition. Once hyperparameters were tuned, scored features consistently outperform the non-scored baseline across all classifiers, achieving up to 98.58% accuracy and 98.81% recall. Recall is the one that matters most in practice. A missed malware sample does not just lower a number on a chart. It means a real threat reached a real device. That is why false negatives carry more weight here than any other error type.

A temporal evaluation of the most recently dated 20% of the existing dataset showed that RFS-MD retained its detection advantage under real-world temporal conditions. Six out of seven classifiers kept their accuracy gains even as malware behavioral drift caused the expected drop in overall performance. On the adversarial side, both the FNN and RF models were put under attack, using PGD and a query-based attack, respectively. In both cases, ASR stayed lower for RFS-MD than for the baseline across every attack strength level tested, and recall gains stayed positive throughout. This confirms the effective resistance against adversarial attacks, making the model more reliable for real-world malware detection scenarios. The comparison against Chi-Square, Information Gain, and Random Forest-based Feature Importance weighting told its own story. For classifiers like LR and NN that are sensitive to how input features are structured, rule-based co-occurrence scoring gave a clear edge. Those same classifiers actually got worse when univariate weighting methods were applied, which points to something specific about how RFS-MD encodes feature relationships that the other methods simply do not capture.

In addition to performance and robustness improvements, the proposed RXAI explainability technique provides transparent and human-understandable insights by capturing meaningful malicious feature patterns; Fidelity results confirm that model decisions align with effective explanation rules. This enhances transparency and trust for malware analysts and addresses an important limitation of black-box ML models in cybersecurity applications.

Overall, the findings highlight the effectiveness of combining rule-based feature scoring and effective explanation rules with ML and DL models to develop a unified framework that bridges accuracy, robustness, and explainability in malware detection systems. Future work may incorporate dynamic analysis features, explore more advanced adversarial attack techniques, and extend the framework to other security domains such as intrusion detection and IoT malware analysis.

https://doi.org/10.3390/electronics15102167

Author Contributions: Conceptualization, A.A. and S.E.; methodology, A.A. and S.E.; software, A.A. and S.E.; validation, A.A. and S.E.; formal analysis, A.A. and S.E.; investigation, A.A. and S.E.; resources, A.A. and S.E.; data curation, A.A. and S.E.; writing—original draft preparation, A.A. and S.E.; writing—review and editing, A.A. and S.E.; visualization, A.A. and S.E.; supervision, A.A. and S.E.; project administration, A.A. and S.E.; funding acquisition, A.A. and S.E. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The AndroZoo dataset utilized in this work is publicly accessible from the official repository at https://androzoo.uni.lu/ (accessed on 2 February 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Jafari, M.; Shameli-Sendi, A. Evaluating the robustness of adversarial defenses in malware detection systems. *Comput. Electr. Eng.* **2026**, *130*, 110845. https://doi.org/10.1016/j.compeleceng.2025.110845.
2. Imran, M.; Appice, A.; Malerba, D. Evaluating Realistic Adversarial Attacks against Machine Learning Models for Windows PE Malware Detection. *Future Internet* **2024**, *16*, 168. https://doi.org/10.3390/fi16050168.
3. Kozak, M.; Demetrio, L.; Trizna, D.; Roli, F. Updating Windows malware detectors: Balancing robustness and regression against adversarial EXEmples. *Comput. Secur.* **2025**, *155*, 104466. https://doi.org/10.1016/j.cose.2025.104466.
4. He, W.; Baig, M.J.A.; Iqbal, M.T. An Internet of Things—Supervisory Control and Data Acquisition (IoT-SCADA) Architecture for Photovoltaic System Monitoring, Control, and Inspection in Real Time. *Electronics* **2025**, *14*, 42. https://doi.org/10.3390/electronics14010042.
5. Poornima, S.; Mahalakshmi, R. Automated malware detection using machine learning and deep learning approaches for android applications. *Meas. Sens.* **2024**, *32*, 100955. https://doi.org/10.1016/j.measen.2023.100955.
6. Rey, V.; Sánchez, P.; Celdrán, A.; Gérôme, B. Federated learning for malware detection in IoT devices. *Comput. Netw.* **2022**, *204*, 108693. https://doi.org/10.1016/j.comnet.2021.108693.
7. Siddiqui, S.; Khan, T. An Overview of Techniques for Obfuscated Android Malware Detection. *SN Comput. Sci.* **2024**, *5*, 328. https://doi.org/10.1007/s42979-024-02637-3.
8. Qiang, W.; Yang, L.; Jin, H. Efficient and Robust Malware Detection Based on Control Flow Traces Using Deep Neural Networks. *Comput. Secur.* **2022**, *122*, 102871. https://doi.org/10.1016/j.cose.2022.102871.
9. Babu, G.; Uma, J. *Adversarial Attacks and Defenses Against Deep Learning in Cybersecurity*; CRC Press: Boca Raton, FL, USA, 2022; pp. 281–296. https://doi.org/10.1201/9781003184140-16.
10. Martins, N.; Cruz, J.; Cruz, T.; Henriques Abreu, P. Adversarial Machine Learning Applied to Intrusion and Malware Scenarios: A Systematic Review. *IEEE Access* **2020**, *8*, 35403–35419. https://doi.org/10.1109/ACCESS.2020.2974752.
11. Lundberg, S.; Lee, S.I. A Unified Approach to Interpreting Model Predictions. *arXiv* **2017**, arXiv:1705.07874.
12. Ribeiro, M.T.; Singh, S.; Guestrin, C. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 13–17 August 2016; KDD '16, pp. 1135–1144. https://doi.org/10.1145/2939672.2939778.
13. Nadeem, A.; Vos, D.; Cao, C.; Pajola, L.; Dieck, S.; Baumgartner, R.; Verwer, S. SoK: Explainable Machine Learning for Computer Security Applications. *arXiv* **2022**. https://doi.org/10.48550/arXiv.2208.10605.
14. Zhang, Z.; Al Hamadi, H.; Damiani, E.; Yeun, C.; Taher, D.F. Explainable Artificial Intelligence Applications in Cyber Security: State-of-the-Art in Research. *IEEE Access* **2022**, *10*, 93104–93139. https://doi.org/10.1109/ACCESS.2022.3204051.
15. Ali, S.; Abuhmed, T.; El-Sappagh, S.; Muhammad, K.; Alonso-Moral, J.M.; Confalonieri, R.; Guidotti, R.; Ser, D.; Díaz-Rodríguez, N.; Herrera, F. Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. *Inf. Fusion* **2023**, *99*, 101805. https://doi.org/10.1016/j.inffus.2023.101805.
16. Saqib, M.; Mahdavifar, S.; Fung, B.C.M.; Charland, P. A Comprehensive Analysis of Explainable AI for Malware Hunting. *ACM Comput. Surv.* **2024**, *56*, 314. https://doi.org/10.1145/3677374.
17. Li, M.Q.; Fung, B.C.; Charland, P.; Ding, S.H. I-MAD: Interpretable malware detector using Galaxy Transformer. *Comput. Secur.* **2021**, *108*, 102371. https://doi.org/10.1016/j.cose.2021.102371.

18. Anthony, P.; Giannini, F.; Diligenti, M.; Homola, M.; Gori, M.; Balogh, S.; Mojzis, J. Explainable Malware Detection with Tailored Logic Explained Networks. *arXiv* **2024**, arXiv:2405.03009.
19. D'Angelo, G.; Ficco, M.; Palmieri, F. Association rule-based malware classification using common subsequences of API calls. *Appl. Soft Comput.* **2021**, *105*, 107234. https://doi.org/10.1016/j.asoc.2021.107234.
20. Kouliaridis, V.; Kambourakis, G. A Comprehensive Survey on Machine Learning Techniques for Android Malware Detection. *Information* **2021**, *12*, 185. https://doi.org/10.3390/info12050185.
21. Senanayake, J.; Kalutarage, H.; Al-Kadri, M.O. Android Mobile Malware Detection Using Machine Learning: A Systematic Review. *Electronics* **2021**, *10*, 1606. https://doi.org/10.3390/electronics10131606.
22. Kauser Sk, H.; Anu V, M. Hybrid deep learning model for accurate and efficient android malware detection using DBN-GRU. *PLoS ONE* **2025**, *20*, e0310230.
23. Muzaffar, A.; Hassen, H.; Lones, M.; Zantout, H. An in-depth review of machine learning based Android malware detection. *Comput. Secur.* **2022**, *121*, 102833. https://doi.org/10.1016/j.cose.2022.102833.
24. Tawfik, M.; Tarazi, H.; Dalalah, A.; Bajeszeyadaljunaeidia; Shaban, W.M.; Fathi, I.S. Few-Shot Android Malware Classification with Quantum-Enhanced Prototypical Learning and Drift Detection. *Sci. Rep.* **2026**, *16*, 10744. https://doi.org/10.1038/s41598-026-45738-0.
25. Akhtar, M.S.; Feng, T. Detection of Malware by Deep Learning as CNN-LSTM Machine Learning Techniques in Real Time. *Symmetry* **2022**, *14*, 2308. https://doi.org/10.3390/sym14112308.
26. Redhu, A.; Choudhary, P.; Srinivasan, K.; Das, T.K. Deep learning-powered malware detection in cyberspace: A contemporary review. *Front. Phys.* **2024**, *12 ,1349463* . https://doi.org/10.3389/fphy.2024.1349463.
27. Bostani, H.; Moonsamy, V. EvadeDroid: A practical evasion attack on machine learning for black-box Android malware detection. *Comput. Secur.* **2024**, *139*, 103676. https://doi.org/10.1016/j.cose.2023.103676.
28. Rafiq, H.; Aslam, N.; Ahmed, U.; Lin, J. Mitigating Malicious Adversaries Evasion Attacks in Industrial Internet of Things. *IEEE Trans. Ind. Inform.* **2022**, *19*, 960–968. https://doi.org/10.1109/TII.2022.3189046.
29. Yu, Z.; Li, S.; Bai, Y.; Han, W.; Wu, X.; Tian, Z. REMSF: A Robust Ensemble Model of Malware Detection Based on Semantic Feature Fusion. *IEEE Internet Things J.* **2023**, *10*, 16134–16143. https://doi.org/10.1109/JIOT.2023.3267337.
30. Kulkarni, M.; Stamp, M. XAI and Android Malware Models. *arXiv* **2024**, arXiv:2411.16817.
31. Nazim, S.; Alam, M.; Rizvi, S.; Mustapha, J.; Hussain, S.; Suud, M. Advancing Malware Imagery Classification with Explainable Deep Learning: A State-of-the-Art Approach Using SHAP, LIME and Grad-CAM. *PLoS ONE* **2025**, *20*, e0318542. https://doi.org/10.1371/journal.pone.0318542.
32. Salih, A.M.; Raisi-Estabragh, Z.; Galazzo, I.B.; Radeva, P.; Petersen, S.E.; Lekadir, K.; Menegaz, G. A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME. *Adv. Intell. Syst.* **2025**, *7*, 2400304. https://doi.org/10.1002/aisy.202400304.
33. Uysal, I.; Köse, U. Analysis of Network Intrusion Detection via Explainable Artificial Intelligence: Applications with SHAP and LIME. In Proceedings of the 2024 Cyber Awareness and Research Symposium (CARS), Grand Forks, ND, USA, 28–29 October 2024. https://doi.org/10.1109/CARS61786.2024.10778742.
34. Manthena, H.; Shajarian, S.; Kimmell, J.C.; Abdelsalam, M.; Khorsandroo, S.; Gupta, M. Explainable Artificial Intelligence (XAI) for Malware Analysis: A Survey of Techniques, Applications, and Open Challenges. *IEEE Access* **2025**, *13*, 61611–61640. https://doi.org/10.1109/access.2025.3555926.
35. Anthony, P.; Galadima, K.R.; Adams, Z.; Onoja, M.; Arp, D.; Homola, M.; Balogh, Š. Rule Extraction and Interaction-Aware Explainability for AI-Driven Malware Detection. In *Proceedings of the Rules and Reasoning*; Hogan, A., Satoh, K., Dağ, H., Turhan, A.Y., Roman, D., Soylu, A., Eds.; Springer: Cham, Switzerland, 2026; pp. 137–155.
36. Alohali, M.A.; Alahmari, S.; Aljebreen, M.; Asiri, M.M.; Miled, A.B.; Albouq, S.S.; Alrusaini, O.; Alqazzaz, A. Two Stage Malware Detection Model in Internet of Vehicles (IoV) Using Deep Learning-Based Explainable Artificial Intelligence with Optimization Algorithms. *Sci. Rep.* **2025**, *15*, 20615. https://doi.org/10.1038/s41598-025-00269-y.
37. Trillo, J.R.; González-López, F.; Morente-Molinera, J.A.; Magán-Carrión, R.; García-Sánchez, P. Evaluation of Explainable, Interpretable and Non-Interpretable Algorithms for Cyber Threat Detection. *Electronics* **2025**, *14*, 3073. https://doi.org/10.3390/electronics14153073.
38. Al-Dujaili, A.; Huang, A.; Hemberg, E.; O'Reilly, U.M. Adversarial Deep Learning for Robust Detection of Binary Encoded Malware. *arXiv* **2018**. https://doi.org/10.48550/arXiv.1801.02950.
39. Slack, D.; Hilgard, S.; Jia, E.; Singh, S.; Lakkaraju, H. Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods. In Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, New York, NY, USA, 7–8 February 2020; AIES '20, pp. 180–186. https://doi.org/10.1145/3375627.3375830.
40. Allix, K.; Bissyandé, T.F.; Klein, J.; Le Traon, Y. AndroZoo: collecting millions of Android apps for the research community. In Proceedings of the 13th International Conference on Mining Software Repositories, New York, NY, USA, 14–15 May 2016; MSR '16, pp. 468–471. https://doi.org/10.1145/2901739.2903508.
41. Alecci, M.; Jiménez, P.J.R.; Allix, K.; Bissyandé, T.F.; Klein, J. AndroZoo: A Retrospective with a Glimpse into the Future. In Proceedings of the 21st International Conference on Mining Software Repositories, New York, NY, USA, 15–16 April 2024; MSR '24, pp. 389–393. https://doi.org/10.1145/3643991.3644863.

42. Curebal, F.; Dağ, H. Enhancing Malware Classification: A Comparative Study of Feature Selection Models with Parameter Optimization. In Proceedings of the 2024 Systems and Information Engineering Design Symposium (SIEDS), Charlottesville, VA, USA, 3 May 2024; pp. 511–516.
43. Han, J.; Pei, J.; Yin, Y.; Mao, R. Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach. *Data Min. Knowl. Discov.* **2004**, *8*, 53–87. https://doi.org/10.1023/B:DAMI.0000005258.31418.83.
44. Lawal, M.; Matthew, O.T. FP-Growth Algorithm: Mining Association Rules without Candidate Sets Generation. *Kasu J. Comput. Sci.* **2024**, *1*, 392–411. https://doi.org/10.47514/kjcs/2024.1.2.0016.
45. Wicaksono, D.; Jambak, M.; Saputra, D. The Comparison of Apriori Algorithm with Preprocessing and FP-Growth Algorithm for Finding Frequent Data Pattern in Association Rule. In *Proceedings of the Sriwijaya International Conference on Information Technology and Its Applications (SICONIAN 2019)*; Atlantis Press: Paris, France, 2020. https://doi.org/10.2991/aisr.k.200424.047.
46. Hahsler, M.; Buchta, C.; Hornik, K. Selective Association Rule Generation. *Comput. Stat.* **2008**, *23*, 303–315. https://doi.org/10.1007/s00180-007-0062-z.
47. Sokolova, M.; Lapalme, G. A systematic analysis of performance measures for classification tasks. *Inf. Process. Manag.* **2009**, *45*, 427–437. https://doi.org/10.1016/j.ipm.2009.03.002.
48. Kim, G.; Yoon, D.; Kwak, N.; Lee, B. Representation-Centric Approach for Android Malware Classification: Interpretability-Driven Feature Engineering on Function Call Graphs. *Appl. Sci.* **2026**, *16*, 2670. https://doi.org/10.3390/app16062670.
49. Jain, S.; Cretu, A.M.; de Montjoye, Y.A. Adversarial Detection Avoidance Attacks: Evaluating the robustness of perceptual hashing-based client-side scanning. *arXiv* **2022**, arXiv:2106.09820.
50. Papernot, N.; McDaniel, P.; Goodfellow, I.; Jha, S.; Celik, Z.B.; Swami, A. Practical Black-Box Attacks against Machine Learning. *arXiv* **2017**, arXiv:1602.02697.
51. Zou, D.; Zhu, Y.; Xu, S.; Li, Z.; Jin, H.; Ye, H. Interpreting Deep Learning-based Vulnerability Detector Predictions Based on Heuristic Searching. *ACM Trans. Softw. Eng. Methodol.* **2021**, *30*, 23. https://doi.org/10.1145/3429444.
52. Tanveer, M.; Munir, K.; Alyamani, H.J.; Hassan, S.; Sheraz, M.; Chuah, T. Graph-augmented multi-modal learning framework for robust android malware detection. *Sci. Rep.* **2025**, *15*, 38341. https://doi.org/10.1038/s41598-025-22169-x.
53. Muzaffar, A.; Hassen, H.R.; Zantout, H.; Lones, M.A. Reassessing Feature-Based Android Malware Detection in a Contemporary Context. *PLoS ONE* **2026**, *21*, e0341013. https://doi.org/10.1371/journal.pone.0341013.
54. Rashid, M.; Qureshi, S.; Abid, A.; Alqahtany, S.; Alqazzaz, A.; Mscs, M.; Al Reshan, M.; Shaikh, A. Hybrid Android Malware Detection and Classification Using Deep Neural Networks. *Int. J. Comput. Intell. Syst.* **2025**, *18*, 52. https://doi.org/10.1007/s44196-025-00783-x.
55. Wei, X.; Cheng, Z.; Li, N.; Lv, Q.; Yu, Z.; Sun, D. DWFS-Obfuscation: Dynamic Weighted Feature Selection for Robust Malware Familial Classification under Obfuscation. *arXiv* **2025**, arXiv:2504.07590. https://doi.org/10.48550/arXiv.2504.07590.
56. Pathak, A.; Barman, U.; Kumar, T.S. Machine learning approach to detect android malware using feature-selection based on feature importance score. *J. Eng. Res.* **2025**, *13*, 712–720. https://doi.org/10.1016/j.jer.2024.04.008.
57. Arrowsmith, J.; Susnjak, T.; Jang-Jaccard, J. Multimodal Deep Learning for Android Malware Classification. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 23. https://doi.org/10.3390/make7010023.
58. Palma, C.; Ferreira, A.; Figueiredo, M. Explainable Machine Learning for Malware Detection on Android Applications. *Information* **2024**, *15*, 25. https://doi.org/10.3390/info15010025.
59. Mitchell, J.; McLaughlin, N.; del Rincon, J.M. Generating sparse explanations for malicious Android opcode sequences using hierarchical LIME. *Comput. Secur.* **2024**, *137*, 103637. https://doi.org/10.1016/j.cose.2023.103637.
60. Soi, D.; Sanna, A.; Maiorca, D.; Giacinto, G. Enhancing android malware detection explainability through function call graph APIs. *J. Inf. Secur. Appl.* **2024**, *80*, 103691. https://doi.org/10.1016/j.jisa.2023.103691.
61. Syed, T.A.; Nauman, M.; Khan, S.; Jan, S.; Zuhairi, M.F. ViTDroid: Vision Transformers for Efficient, Explainable Attention to Malicious Behavior in Android Binaries. *Sensors* **2024**, *24*, 6690. https://doi.org/10.3390/s24206690.
62. Alani, M.M.; Mashatan, A.; Miri, A. XMal: A lightweight memory-based explainable obfuscated-malware detector. *Comput. Secur.* **2023**, *133*, 103409. https://doi.org/10.1016/j.cose.2023.103409.
63. Shokouhinejad, H.; Razavi-Far, R.; Higgins, G.; Ghorbani, A.A. Enhancing GNN Explanations for Malware Detection with Dual Subgraph Matching. *Mach. Learn. Knowl. Extr.* **2026**, *8*, 2. https://doi.org/10.3390/make8010002.
64. Li, F.; Wang, S.; Liew, A.W.C.; Ding, W.; Liu, G. Large-Scale Malicious Software Classification with Fuzzified Features and Boosted Fuzzy Random Forest. *IEEE Trans. Fuzzy Syst.* **2020**, *29*, 3205–3218. https://doi.org/10.1109/TFUZZ.2020.3016023.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.